



ULPGC

**Universidad de
Las Palmas de
Gran Canaria**

**Escuela de
Ingeniería Informática**



Recetas de la Abuela

TITULACIÓN: Grado en Ingeniería Informática – Mención de
Tecnologías de la Información

AUTOR: Pablo Alonso Galván

TUTORIZADO POR:

Agustín Trujillo Pino

Raúl Mendoza Álamo

Fecha: Junio de 2022

DIRECTORA DE LA ESCUELA DE INGENIERÍA INFORMÁTICA

Agradecimientos

Doy las gracias a mis amigos por siempre haber estado ahí dándome apoyo, tanto en los pequeños momentos como en las instancias difíciles; momentos de estrés y presión, por haber servido de inspiración y por la compañía ofrecida durante estos años de carrera.

Agradezco también a toda mi familia que me ha educado y ayudado a tomar las decisiones, desde las más grandes hasta las más mundanas, que hacen que hoy en día sea quien soy me encuentre donde estoy.

Por último, a mi tutor Agustín por los consejos y el tiempo que ha dedicado en la ayuda a la creación de este proyecto, y a Gustavo de Singular por la confianza que ha depositado en mí.

Resumen

Hoy en día, con el auge de la tecnología móvil y la telecomunicación, la capacidad de compartir y adquirir conocimientos nunca había sido tan fácil.

Sin embargo, existe un colectivo de personas como es el de la tercera edad, que, por falta de adaptación al rápido cambio que presenta dichas tecnologías, muestra dificultad a la hora de acceder a ellas, lo que en muchos casos impide que conocimientos y experiencias recolectados durante toda una vida sean divulgados más allá de algunos familiares o conocidos.

Éste trabajo se centra en el campo de dichos conocimientos referente a la cocina y gastronomía tradicional. “Las recetas de la abuela” es una aplicación móvil que tiene como objetivo el actuar como recetario y que servirá de herramienta para ayudar a difundir el saber de las personas mayores a los más jóvenes.

Abstract

Nowadays, with the rise of mobile technology and telecommunications, the ability to share and acquire knowledge has never been so easy.

However, there is a group of people, such as the elderly, who, due to a lack of adaptation to the rapid changes in these technologies, find it difficult to access them, which in many cases prevents knowledge and experiences collected over a lifetime from being disseminated beyond a few relatives or acquaintances.

This work focuses on the field of traditional cooking and gastronomy. “Grandma's recipes” is a mobile application that aims to act as a recipe book and will serve as a tool to help disseminate the knowledge of the elderly to younger people.

Tabla de contenido

1.	Introducción.....	11
1.1	Motivación.....	11
1.2	Objetivos.....	12
1.2.1	Objetivos Académicos.....	12
1.2.2	Objetivos Personales.....	12
1.3	Estado actual	13
1.4	Estructura del documento	15
2.	Competencias específicas y aportaciones.....	17
2.1	Competencias específicas cubiertas.....	17
2.1.1	TI01	17
2.1.2	TI06	17
2.1.3	CII01.....	17
2.1.4	CII017.....	17
2.1.5	TFG01.....	17
2.2	Aportaciones.....	18
2.2.1	Aportaciones al entorno económico y social.....	18
2.2.2	Aportaciones personales	18
3.	Metodología y planificación del proyecto	19
3.1	Metodología.....	19
3.2	Planificación del proyecto	20
4.	Recursos empleados	21
4.1	Hardware	21
4.2	Software.....	21
4.2.1	Herramientas.....	21
4.2.2	Tecnologías.....	23
4.2.3	<i>Frameworks</i> y Herramientas Relacionadas	23
5.	Normativa y legislación.....	26
5.1	Licencias.....	26
5.1.1	Licencia MIT	26
5.1.2	Licencia Pública General de GNU	26
5.1.3	Microsoft 365 para estudiantes.....	26
5.1.4	Licencias de imágenes	27
5.2	Legislación respecto al tratamiento de datos	27

6.	Análisis	28
6.1	Actores.....	28
6.1.1	Invitado.....	28
6.1.2	Usuario registrado	28
6.2	Casos de uso	29
7.	Diseño.....	30
7.1	Arquitectura del sistema	30
7.2	Interfaz de usuario	31
7.2.1	Elementos comunes	32
7.2.2	Inicio y búsqueda.....	33
7.2.3	Vista detallada de una receta	35
7.2.4	Vistas de autenticación.....	36
7.2.5	Creación y edición de recetas	38
7.2.6	Creación y edición del paso de una receta.....	39
7.2.7	Mis recetas y favoritos.....	40
7.2.8	Principios del software utilizados.....	41
7.3	Fujo de la aplicación.....	42
7.4	Diseño del <i>Backend</i>	43
8.	Desarrollo	45
8.1	Desarrollo del <i>backend</i>	45
8.1.1	Usando el modelo RESTful.....	45
8.1.2	Estructura general del proyecto	46
8.1.3	Implementación de <i>Doctrine</i>	47
8.1.4	Autenticación y Seguridad	48
8.1.5	Otras librerías y herramientas configuradas	50
8.1.6	Controladores y métodos de manejo de llamadas	52
8.2	Desarrollo del <i>FrontEnd</i>	53
8.2.1	Estructura general de la aplicación.....	53
8.2.2	Módulos utilizados	56
8.2.3	Navegación	57
8.2.4	Estilo de las Peticiones	60
8.3	Pruebas	62
9.	Conclusiones y trabajo futuro.....	63
9.1	Resultados y Conclusiones	63

9.2 Dificultades encontradas.....	63
9.3 Mejoras Futuras	64
10. Manual de usuario	65
11. Referencias	77

Tabla de Figuras

Figura 1: Entrevistados españoles que afirman tener grandes conocimientos con la comida y la cocina [55]	11
Figura 2: Logo de Cookpad	13
Figura 3: Logo de MyRealFood	13
Figura 4: Logo de Tasty	14
Figura 5: El "modelo cascada" sin modificar.	19
Figura 6: Diagrama de casos de uso	29
Figura 7: Ilustración de un modelo cliente-servidor.....	30
Figura 8: Modelo de cliente-servidor de 3 capas.....	31
Figura 9: Paleta de colores de la aplicación	32
Figura 10: Barra superior en pantalla principal	32
Figura 11: Barra superior en vista superpuesta	32
Figura 12: Ejemplo de modal de selección de imágenes.....	33
Figura 14: Página principal.....	33
Figura 13: Tarjeta de información de receta en el listado	33
Figura 16: Menú lateral de usuario dado de alta	34
Figura 15: Menú lateral de usuario invitado.....	34
Figura 17: Ejemplo de búsqueda	35
Figura 18: Vista completa de una receta II.....	36
Figura 19: Vista completa de una receta I	36
Figura 20: Ejemplo de diálogo de error.....	37
Figura 21: Pantalla de registro.....	37
Figura 22: Inicio de sesión	37
Figura 23: Segundo paso de creación de una receta	38
Figura 24: Primer paso de creación de una receta	38
Figura 25: Tercer paso de creación de una receta	39
Figura 26: Vista de creación de un paso de elaboración	39
Figura 27: Vista de recetas de usuario.....	40
Figura 28: Diálogo de confirmación para eliminar una receta	41
Figura 29: Visión general del flujo de la aplicación.....	43
Figura 30: Diagrama de base de datos.....	44
Figura 31: Comando de generación del proyecto de Symfony	45
Figura 32: Estructura general del proyecto.....	47

Figura 33: Entidades del proyecto generadas por Doctrine	48
Figura 34: Diagrama de ejemplo de flujo de trabajo de Doctrine [56].....	48
Figura 35: Archivo security.yaml en la configuración del firewall.....	50
Figura 36: Ejemplo de serializer de Usuario	50
Figura 37: Form Type de las recetas	51
Figura 38: Ejemplo de creación de receta sin FormType.....	51
Figura 39: Ejemplo de creación de receta con FormType.....	51
Figura 40: Métodos por defecto del repositorio de receta	52
Figura 41: Función de borrar receta.	53
Figura 42: Estructura general del proyecto.....	54
Figura 43: Estructura del directorio /src	55
Figura 44: Estructura del directorio /src/modules	55
Figura 45: Identificadores de las rutas en el archivo Routing.js	58
Figura 46: Función personalizada de navegación a la vista de detalles de receta	58
Figura 47: Acciones generales de navegación.....	59
Figura 48: Escenas de navegación en Router.js	60
Figura 49: Función de llamada a la API para la creación de una receta.....	61
Figura 50: Pantalla principal.....	65
Figura 51: Resultado de la búsqueda.....	66
Figura 52: Búsqueda de recetas	66
Figura 53: Pantalla de detalles de la receta.....	67
Figura 54: Información final de la vista detallada de la receta.....	68
Figura 55: Modal de puntuación de receta	68
Figura 56: Modal de compartir por medios externos	68
Figura 57: Formulario de registro.....	69
Figura 58: Menú desplegable (Drawer).....	69
Figura 59:Ejemplo del formulario de registro completado	70
Figura 60: Ejemplo de fallo de la validación de registro.....	70
Figura 61: Inicio de sesión	71
Figura 62: Favoritos	72
Figura 63: Editar Perfil	72
Figura 64: Drawer con sesión iniciada	72
Figura 65: Cerrar sesión	72
Figura 66: Recetas del usuario.....	73

Figura 67: Dejar comentario.....	73
Figura 68: Receta añadida a favoritos	74
Figura 69: Corazón de favoritos	74
Figura 70: Paso 2 creación de receta	74
Figura 71: Paso 1 creación de receta	74
Figura 72: Creación de paso nuevo	75
Figura 73: Paso 3 creación de receta	75
Figura 74: Vista de "Mis Recetas"	76
Figura 75: Formulario de edición de receta.....	76
Figura 76: Mensaje de confirmación de borrado de receta	76

1. Introducción

1.1 Motivación

Las recetas de la abuela es una aplicación comisionada por la empresa Singular Factory, y su fundador Gustavo Medina. Una experiencia que marcó la motivación del Fundador de Singular Factory a ofrecer esta idea como base para un proyecto es el pensamiento que tuvo una vez de:

“Nunca más volveré a comer unas almejas como las que hacía mi madre”.

La aplicación viene motivada por el hecho de que las nuevas generaciones, en su mayoría, no poseen tiempo, ganas, o las herramientas para aprender a cocinar, y mucho del saber de las “abuelas” se pierde una vez que ya no se encuentran entre nosotros.

Acorde con recientes datos y estadísticas, España entra dentro de los países donde los jóvenes de entre 20 y 29 años afirman tener menos conocimientos culinarios y horas dedicadas a la cocina, siendo solo un 20% los que afirman tener un gran conocimiento y experiencia, y un 27% los que declaran sentir verdadera pasión [1].

Siendo España un país con una cultura gastronómica tan reconocida y extensa, es una pena que se pierda tan fácilmente una parte de lo que define a los españoles. Con las Recetas de la Abuela, se espera paliar de alguna manera esta pérdida de patrimonio y dar a conocer recetas familiares con mucha historia y así incitar el resurgir de la comida más tradicional en los hogares. Además, pretende valerse del surgimiento de la cultura “Realfooding”, que promueve el consumo de alimentos que han sido mínimamente procesados o alterados y por ello, se evitan los productos ultra procesados a toda costa [2], ya que no existe comida que mejor se adapte a esta filosofía que la comida tradicional.

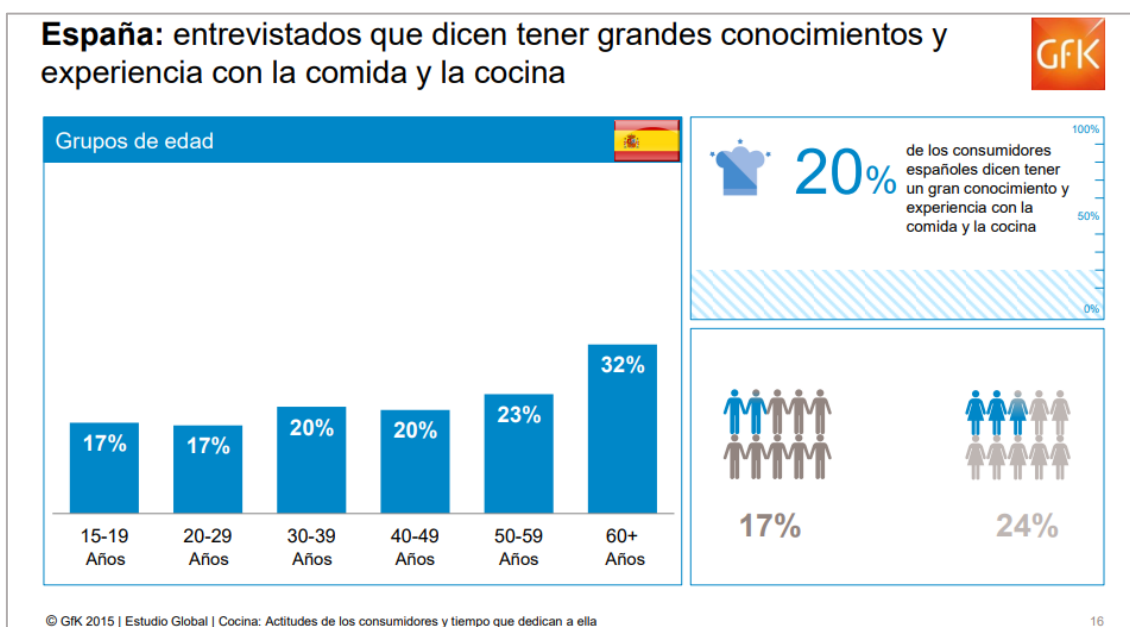


Figura 1: Entrevistados españoles que afirman tener grandes conocimientos con la comida y la cocina [55]

1.2 Objetivos

1.2.1 Objetivos Académicos

El proyecto de las recetas de la abuela planea mejorar frente a un proyecto parecido ya entregado como TFG anteriormente, también bajo la supervisión de Singular Factory, de forma que esté más acorde con la visión general que planean. Principalmente ofrecerá diferencias en el *Backend* de la aplicación y conexión con el servidor.

El servicio ofrecido por la aplicación móvil tiene como objetivo ofrecer dos posibles perfiles de usuarios. El primero es el de cliente potencial de la aplicación, que podrá ver y buscar recetas en la plataforma, además de darse de alta en el servicio.

El segundo perfil de usuario, creador de contenido (abuelas con ayuda de sus nietos o hijos) o usuario ya registrado, será capaz de generar recetas, aparte de poder ver las ya creadas. Para ello proporcionará la información necesaria a la preparación del plato, mostrando ingredientes y detallando los diferentes pasos a seguir, ayudándose de imágenes y vídeos. Los usuarios registrados pueden guardar recetas en favoritos. Además, para dar a conocer las mejores recetas, estos usuarios podrán votar, comentar y compartir aquellas que más les agraden.

1.2.2 Objetivos Personales

Este proyecto me plantea varias metas personales, como son el adquirir los conocimientos y la experiencia necesarias para ser capaz de construir una aplicación móvil prácticamente desde cero, además de generar nuevas aptitudes y manejo con herramientas y tecnologías que me eran desconocidas hasta ahora, de las cuales ya se hablará con más detalle más adelante.

Asimismo, a nivel profesional, desarrollar competencias en el diseño de aplicaciones para el mercado móvil, un sector que se encuentra en auge y se prevé que continúe creciendo, creo que puede enriquecer mucho mis prospectos para el mercado laboral, además de las habilidades de organización y responsabilidad que ofrece el desarrollo del trabajo de final de título y el manejo en un entorno laboral real gracias a que ha sido desarrollado para Singular Factory.

1.3 Estado actual

En este apartado de la memoria se ha realizado un pequeño estudio de las aplicaciones en el mercado que pudiesen existir que se ajusten a la temática del trabajo, siendo estas aplicaciones móviles orientadas a ofrecer y compartir recetas de cocina saludable y tradicional, en especial recetas pertenecientes a personas mayores.

Para obtener resultados, se ha realizado una búsqueda de aplicaciones más populares con la palabra clave “recetas” de la categoría “comer y beber” en la plataforma de mercado de *Apps* propietario de la empresa Google: *Google Play Store* [3]. De aquí se han obtenido varios resultados que, a pesar de no ajustarse del todo a la temática del trabajo, ya que la mayoría son orientadas a simplemente recetas de cocina saludable, creo que podría ayudar al desarrollo del trabajo y a hacerse un poco a la idea de cómo está el estándar del sector si analizamos algunas de éstas.

Cookpad [4]:

Recetario de comida casera, donde los usuarios pueden ver y crear recetas, explicando paso por paso, ayudándose de imágenes, para compartir entre ellos. El resto de los usuarios pueden votar, compartir y comentar dichas recetas, además de consejos e ideas. Esta es la aplicación más popular de esta categoría, con más de 10 millones de descargas.

Popularidad: +10 millones de descargas.

Valoración: 4.7/5 (292k votos)



Figura 2: Logo de Cookpad

- MyRealFood [5]:

Aplicación centrada a planificación de la comida de la semana, con un enfoque a recetas saludable. Surge tras la popularización del movimiento “RealFooder”. Esta aplicación es más completa en lo que se refiere a funcionalidades como el manejo de calorías en el plato y los ingredientes, además de un carrito de la compra, además de el enfoque de red social culinario que plantea.

Popularidad: +1 millones de descargas.

Valoración: 4.6/5 (27k votos)



Figura 3: Logo de MyRealFood

- Tasty [6]:

Esta aplicación es diferente al resto en el sentido de que, a pesar de que también ofrece recetas sanas y con opciones veganas y vegetarianas, un usuario solo puede acceder y guardar las recetas creadas por la propia Tasty, no las de otros usuarios. Básicamente se plantea como una base de datos de recetas propietarias, lo que difiere mucho de las anteriores y la de este proyecto.

Popularidad: +10 millones de descargas.

Valoración: 4.4/5 (162k votos)



Figura 4: Logo de Tasty

Estas aplicaciones presentan versiones con funciones parecidas a lo que se quiere lograr con Las Recetas de la Abuela, siendo quizás la más completa *Cookpad*, ya que ofrece un sistema de comunidad muy integrado. Sin embargo, tal y como se mencionó antes, estas aplicaciones echan en falta el enfoque tradicional y la relación con la cultura gastronómica que quiere ofrecer este proyecto. Además, dichas aplicaciones no afrontan el verdadero problema que pretendemos solucionar, que es el que las personas mayores, que mantienen la mayoría de conocimientos y trucos de este estilo culinario, tienen una gran falta de accesibilidad a estas tecnologías, cosa que se quiere aplacar en Las Recetas de la Abuela, ofreciendo un sistema fácil de usar y orientado a que los hijos y nietos de éstas personas ayuden a sus mayores a compartir estos conocimientos, ofreciendo una experiencia de usuario divertida a la par que didáctica.

1.4 Estructura del documento

Pasada la introducción del proyecto, el trabajo presentado se encuentra repartido en diferentes secciones en función del contenido que presentan. En este apartado se procederá a explicar con cierto detalle cada una de ellas:

- Competencias específicas y aportaciones

En este capítulo se enumerarán y desarrollarán las diferentes competencias trabajadas en este proyecto que estén relacionadas con las aportadas en el Grado en Ingeniería Informática. También se describirán las diferentes aportaciones resultado del proyecto.

- Metodología y planificación

Aquí se presentarán la metodología utilizada en el desarrollo del proyecto y la planificación seguida.

- Recursos empleados

En este apartado del documento se definirán los recursos y tecnologías utilizados durante el desarrollo del proyecto, tanto *hardware* como *software*.

- Normativa y Legislación

Aquí se desarrollará e informará sobre las licencias empleadas por el autor para la utilización de ciertas herramientas del proyecto, además de normativas requeridas por la aplicación.

- Análisis

Apartado en el que se tocará la fase de análisis del proyecto, donde se comentarán los requisitos de la aplicación y los distintos actores que harán uso de ella mediante diagramas de casos de uso.

- Diseño

Apartado relacionado con la fase de diseño del proyecto, donde se explicarán los principios utilizados en el diseño, la arquitectura planteada, y la planificación tanto del *backend* como del *frontend*.

- Desarrollo

En este capítulo se explicará la fase de desarrollo, donde se tocará la estructura de ambas capas de la aplicación, las librerías empleadas en el proyecto, así como la conexión con la API y el tratamiento de peticiones y respuestas. Al final se mostrará también el procedimiento seguido para la fase de pruebas del producto.

- Conclusiones y trabajo futuro

Aquí se comentarán los resultados finales obtenidos del proyecto, además de desarrollar las conclusiones obtenidas a partir de su diseño y desarrollo. Finalmente se tratarán las funcionalidades que se puedan añadir en un futuro.

- Manual de usuario

Capítulo en el que se presenta la aplicación final explicada para que un usuario recién llegado sea capaz de navegar por ella y acceder a todas sus funcionalidades.

- Bibliografía

Apartado final del proyecto, donde se recogen las referencias bibliográficas empleadas durante la redacción de este documento.

2. Competencias específicas y aportaciones

2.1 Competencias específicas cubiertas

La información presentada en este apartado ha sido obtenida del plan de proyecto docente del Grado de Ingeniería Informática de la Universidad de las Palmas de Gran Canaria [7].

2.1.1 TI01

“Capacidad para comprender el entorno de una organización y sus necesidades en el ámbito de las tecnologías de la información y las comunicaciones.”

Al haberse desarrollado con la organización Singular Factory, se han establecido reuniones a lo largo del desarrollo en las que se ha declarado el funcionamiento de la empresa por dentro, como está organizada, su ámbito de trabajo y las necesidades de la aplicación.

2.1.2 TI06

“Capacidad de concebir sistemas, aplicaciones y servicios basados en tecnologías de red, incluyendo Internet, web, comercio electrónico, multimedia, servicios interactivos y computación móvil.”

Este proyecto tiene como objetivo ofrecer una aplicación móvil, además de los servicios *backend* utilizados para que funcione correctamente el manejo de datos y la interacción entre el usuario y el servidor.

2.1.3 CII01

“Capacidad para diseñar, desarrollar, seleccionar y evaluar aplicaciones y sistemas informáticos, asegurando su fiabilidad, seguridad y calidad, conforme a principios éticos y a la legislación y normativa vigente.”

Esta competencia es aplicable a las diferentes fases del proyecto. Tal como se describirá en el apartado de desarrollo, se han aplicado las fases de diseño, desarrollo y pruebas para la correcta compleción del proyecto. Además, se ha tenido siempre en cuenta la seguridad y fiabilidad del producto como filosofía.

2.1.4 CII017

“Capacidad para diseñar y evaluar interfaces persona computador que garanticen la accesibilidad y usabilidad a los sistemas, servicios y aplicaciones informáticas.”

Debido al motivo del trabajo, uno de los objetivos es desarrollar una interfaz de usuario que se adapte a la necesidad de las personas mayores, a las que más les cuesta navegar y adaptarse a las nuevas tecnologías.

2.1.5 TFG01

“Ejercicio original a realizar individualmente y presentar y defender ante un tribunal universitario, consistente en un proyecto en el ámbito de las tecnologías específicas de la Ingeniería en Informática de naturaleza profesional en el que se sinteticen e integren las competencias adquiridas en las enseñanzas.”

La realización del trabajo de fin de grado, siendo éste un conjunto de el desarrollo de la aplicación, redacción del informe final y posterior defensa ante el tribunal asignado.

2.2 Aportaciones

2.2.1 Aportaciones al entorno económico y social

El mercado de dispositivos móviles lleva en auge desde la invención del *SmartPhone* o teléfono inteligente. Además, se encuentra cada vez más integrado en cada aspecto de nuestra vida cotidiana. Esto implica una capacidad de mercado bastante grande. Además, este proyecto propone expandir este negocio al grupo de población que menos hace uso de esta tecnología, como son las personas mayores, a base de crear una aplicación fácil de usar o que pueda emplearse en conjunto con sus hijos o nietos, lo que abre la posibilidad de ofrecer experiencia y soltura con los dispositivos móviles. Esto además podría ayudar a aumentar la capacidad de las personas mayores de integrar dichos dispositivos en el día a día, no solo para con la aplicación presentada. Todo ello por supuesto conlleva grandes ventajas.

2.2.2 Aportaciones personales

El proyecto me ha ayudado a adquirir cierta capacidad de organización y responsabilidad. Gracias a que se ha desarrollado para Singular Factory, he tenido la posibilidad de experimentar como es un entorno de trabajo real, además de contar con la guía y la experiencia de personal muy cualificado en el entorno de desarrollo de aplicaciones para el mercado móvil. También me ha aportado experiencia en el desarrollo de aplicaciones software, en concreto, de dispositivos móviles, desde cero y en las diferentes fases del desarrollo del producto, como la planificación, desarrollo y pruebas. Además, me ha servido para poner en práctica en un entorno más profesional los conocimientos adquiridos durante este periodo de estudios en la universidad, desde el desarrollo software hasta creación y gestión de bases de datos.

3. Metodología y planificación del proyecto

3.1 Metodología

Para el desarrollo de este proyecto, se ha buscado una metodología que se adapte a la situación de desarrollo que presenta, siendo un Trabajo de Fin de Título. Dado que la idea del proyecto está planteada por una empresa, que tiene una visión clara del producto, y va a ser desarrollada únicamente por una persona, desde su fase inicial de planificación hasta la entrega de un producto mínimo viable, se ha escogido una metodología de tipo cascada.

Más adelante se hará más hincapié en lo referente a cada fase en el apartado de planificación, y se explicará en profundidad cada fase en los apartados posteriores del documento dedicados a ello. Una metodología de desarrollo en cascada es aquella que se caracteriza por dividir los procesos de desarrollo en sucesivas fases de proyecto. Al contrario que en los modelos iterativos, cada una de estas fases se ejecuta tan solo una vez. Los resultados de cada una de las fases sirven como hipótesis de partida para la siguiente. [8]

Los modelos más habituales de las fases de desarrollo en cascada son los modelos que dividen los procesos de desarrollo en cinco fases. En la práctica, se pueden aplicar diferentes versiones del modelo. Un ejemplo de modelo estándar, con sus 5 fases puede ser:

- Análisis: planificación, análisis y especificación de los requisitos.
- Diseño: diseño y especificación del sistema.
- Implementación: programación y pruebas unitarias.
- Verificación: integración de sistemas, pruebas de sistema y de integración.
- Mantenimiento: entrega, mantenimiento y mejora.

En nuestro caso, las fases se han reducido de manera que se han ajustado a las planteadas en un primer momento en el documento de planificación del proyecto (TFT01), que se encuentra adjunto en este mismo informe. Así, se presentan fases de análisis, diseño, validación y documentación.

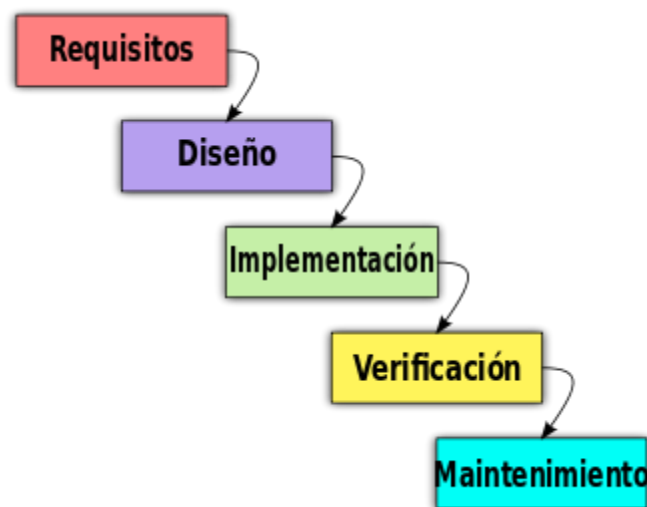


Figura 5: El "modelo cascada" sin modificar.

3.2 Planificación del proyecto

Las diferentes fases del desarrollo del proyecto fueron planteadas en un primer momento durante la planificación de éste en el documento TFT01, donde se especificaron las diferentes tareas a realizar durante el desarrollo del trabajo, además de una estimación de la duración de cada una de ellas. El resultado final de la planificación quedó en:

- Fase de Estudio previo / Análisis (Duración estimada de 60 horas):

En esta fase de estudio previo, a base de reunirse con el cliente, en este caso, Singular Factory, se analizaron y se dejaron claros los diferentes requisitos que iba a necesitar la aplicación, además de tratar con las tecnologías necesarias para el desarrollo de ésta. Ya que el proyecto se iba a realizar sobre una base, y lo que se plantea en este trabajo es la mejora de un trabajo anterior, se designaron los cambios y objetivos y se propusieron ideas nuevas propias que podía ofrecer al proyecto, además de tecnologías nuevas que se pudiesen implementar.

- Fase de Diseño/ Desarrollo/ Implementación (Duración estimada de 180 horas):

Esta fase es claramente la más larga de todo el conjunto, ya que aquí se comienza a preparar el entorno de trabajo, la instalación de las diferentes tecnologías y herramientas necesarias y diseñar los diferentes componentes de la aplicación, como son la arquitectura del software y la base de datos, además del flujo de la aplicación *FrontEnd* e interfaz de usuario y la lógica del *BackEnd*. Posterior a esto, se empieza a implementar todos los diseños, lo que ocupa una gran parte del tiempo.

- Evaluación/ Validación/ Prueba (Duración estimada de 30 horas):

Última fase del desarrollo del producto del proyecto, la aplicación. En esta fase se pone a prueba la aplicación simulando diferentes situaciones en el flujo de uso de ésta a base de pruebas internas y con el uso del producto por los usuarios de la empresa.

- Documentación/ Presentación (Duración estimada de 30 horas):

Esta es la última fase del desarrollo del proyecto completo, en el que se desarrolla la documentación y la redacción de la memoria, siendo este propio documento, para la posterior defensa ante un tribunal.

4. Recursos empleados

En este apartado del proyecto se describirán los recursos empleados y el razonamiento de la elección, desde el hardware como las herramientas y tecnologías software.

4.1 Hardware

- Ordenador portátil HP 15-bwx00.
 - Procesador AMD A10-9620P RADEON R5 @ 2.5 GHz x 4 Núcleos
 - Arquitectura de 64 bits.
 - 12 GB de memoria RAM
 - Sistema operativo Windows 10.
- Teléfono Móvil Huawei P8 lite 2017
 - Sistema operativo Android 8
- Teléfono Móvil Huawei P Smart
 - Sistema operativo Android 9

4.2 Software

4.2.1 Herramientas

Conjunto de herramientas empleadas durante la dirección del proyecto.

- **Trello**

Trello es un software de oficina diseñado para la gestión de actividades y la administración de proyectos de manera colectiva. Este programa informático está optimizado para organizar información en formatos visuales simples que facilitan la realización de tareas y el cumplimiento de objetivo [9]. Para este proyecto se le ha dado el uso de dividir y gestionar las tareas a llevar a cabo y organizarlas en diferentes campos, como pendientes, en proceso y, posteriormente, completadas.

- **Figma**

Figma es una aplicación de edición de gráficos y diseño de interfaces de usuario basada en la web. Se puede utilizar para realizar todo tipo de trabajos de diseño gráfico, desde la creación de esquemas de sitios web, el diseño de interfaces de aplicaciones móviles, la creación de prototipos de diseños, o elaboración de publicaciones en las redes sociales [10]. En este proyecto se ha utilizado para diseñar *mockups* de las interfaces de usuario, basándose en diseños proporcionados por Singular Factory.

- **Postman**

Se trata de una herramienta dirigida a desarrolladores web que permite realizar peticiones HTTP a cualquier API. *Postman* es muy útil a la hora de programar y hacer pruebas, puesto que nos ofrece la posibilidad de comprobar el correcto funcionamiento de nuestros desarrollos [11].

- **Microsoft Visual Studio Code**

IDE – Integrated Development Environment (Entorno de desarrollo integrado) y editor de código fuente independiente que se ejecuta en *Windows*, *macOS* y *Linux*. La elección principal para desarrolladores web y JavaScript, con extensiones para admitir casi cualquier lenguaje de programación [12]. Es un programa de código abierto fundado y patrocinado por *Microsoft*. Contiene gran cantidad de integraciones con otras tecnologías muy útiles para el desarrollo del proyecto, como por ejemplo *Git*.

- **Android Studio**

IDE oficial de aplicaciones para el entorno *Android*. Creado y financiado por *Google*, ofrece una serie de funciones y características centradas en el desarrollo de proyectos en *Android*, como pueden ser las *Android SDK - Software Development Tools* o un emulador donde instalar y probar las aplicaciones [13].

- **GitHub**

GitHub es una plataforma de desarrollo colaborativo para alojar proyectos utilizando el sistema de control de versiones *Git*. Se utiliza principalmente para la creación de código fuente de programas de ordenador y ofrece gestión de repositorios y versiones [14].

- **XAAMP**

XAMPP es un servidor web multiplataforma que es gratuito y de código abierto. Las siglas *XAMPP* son un acrónimo de *Cross-Platform*, *Apache*, *MySQL*, *PHP* y *Perl*. Es un popular servidor web multiplataforma que permite a los programadores escribir y probar su código en un servidor web local. Incluye *MariaDB*, *Apache HTTP Server* e intérpretes para *PHP* y *Perl*, entre otros lenguajes informáticos [15].

- **PhpMyAdmin**

PhpMyAdmin es una aplicación web que sirve para administrar bases de datos *MySQL* de forma sencilla y con una interfaz amistosa. Se trata de un software muy popular basado en *PHP*. La ventaja de usar una aplicación web es que nos permite conectarnos con servidores remotos, a los cuales no siempre se puede acceder usando programas de interfaz gráfica [16]. Viene integrado en la descarga de *XAAMP*.

- **MariaDB**

MariaDB es un sistema de gestión de bases de datos derivado de *MySQL* de código libre con licencia *GPL (General Public License)*. Tiene una alta compatibilidad con *MySQL* ya que posee las mismas órdenes, interfaces, *API* y bibliotecas, siendo su objetivo poder cambiar un servidor por otro directamente [17].

- **DB Designer**

DBDesigner es un software de diseño gráfico de bases de datos que integra todos los niveles de diseño, modificación, creación y mantenimiento en la estructura de una base de datos. Combina una interfaz fácil de usar con potentes herramientas que permiten la generación rápida de scripts *SQL* o *XML* para crear las bases de datos diseñadas [18].

- **Microsoft 365**

Conjunto de herramientas de la *Suite Office* para empresas y particulares utilizadas para el desarrollo del informe y la presentación. Las aplicaciones he las que se ha hecho uso durante el transcurso del proyecto son *Word* y *PowerPoint* [19].

- **Brave Browser**

Brave es un navegador web gratuito y de código abierto desarrollado por Brave Software, Inc. basado en el navegador web *Chromium*. Es un navegador centrado en la privacidad, que bloquea automáticamente los anuncios en línea y los rastreadores de sitios web en su configuración predeterminada [20]. Durante el desarrollo del proyecto, se ha dado uso de las herramientas de consola de *Chrome* para el depurado de la aplicación.

4.2.2 Tecnologías

- **Git**

Git es un software libre de control de versiones, pensando en la eficiencia, la confiabilidad y compatibilidad del mantenimiento de versiones de aplicaciones cuando estas tienen un gran número de archivos de código fuente. Herramientas como *GitHub* y *GitLab* están basadas en esta tecnología [21].

- **PHP**

PHP (acrónimo recursivo de *PHP: Hypertext Preprocessor*) es un lenguaje de código abierto muy popular especialmente adecuado para el desarrollo web y que puede ser incrustado en *HTML*. Lo que distingue a *PHP* de algo del lado del cliente como *Javascript* es que el código es ejecutado en el servidor, generando *HTML* y enviándolo al cliente. El cliente recibirá el resultado de ejecutar el script, aunque no se sabrá el código subyacente que era [22].

- **JavaScript**

Es un lenguaje de secuencias de comandos orientado a objetos que permite crear contenido de actualización dinámica muy popular en el desarrollo web, principalmente en el lado del cliente [23].

- **NodeJS**

Node.js es un entorno de ejecución de JavaScript de código abierto y multiplataforma que ejecuta código JavaScript fuera de un navegador web. *Node.js* permite a los desarrolladores utilizar *JavaScript* para escribir herramientas de línea de comandos y para la ejecución de *scripts* del lado del servidor con el fin de producir contenidos dinámicos para las páginas web antes de que se envíen al navegador del usuario [24].

- **NPM- Node Package Manager**

Npm es en primer lugar, un repositorio en línea para la publicación de proyectos *Node.js* de código abierto, además de una utilidad de línea de comandos para interactuar con dicho repositorio que ayuda a la instalación de paquetes, la gestión de versiones y la gestión de dependencias [25].

4.2.3 Frameworks y Herramientas Relacionadas

En este apartado se explicarán los *frameworks* o entornos de trabajo estandarizados utilizados, y sus herramientas relacionadas. Además, se desarrollará brevemente las principales ventajas de haber escogido cada uno de ellos para el desarrollo del proyecto.

- **React Native**

React Native es un *framework JavaScript* para crear aplicaciones reales nativas para *iOS* y *Android*, basado en la librería de *JavaScript React* para la creación de componentes visuales, cambiando el propósito de estos para, en lugar de ser ejecutados en navegador, correr directamente sobre las plataformas móviles nativas, en este caso *iOS* y *Android*. Es decir, en lugar de desarrollar una aplicación web híbrida o en HTML5, lo que obtienes al final como resultado es una aplicación real nativa, indistinguible de la que podrías desarrollar con tu código en *Objective-C* o *Java* [26].

A continuación, se comentarán algunas de las ventajas de *React Native* que han ayudado a su elección para el desarrollo del *FrontEnd* proyecto.

1. **Compatibilidad Cross Platform:** Las aplicaciones creadas usando este framework son compiladas tanto en *Android* como en *iOS*. Al usar *React Native*, solo se necesita compilar una aplicación una vez en *JavaScript*. Sin embargo, seguir un enfoque nativo requiere crear dos aplicaciones por separado para *iOS* y *Android* desde cero. Dado que *React Native* necesita que escriba solo un código, no se necesitan dos equipos de desarrolladores. Esto abarata mucho el coste necesario para la construcción y el mantenimiento de la aplicación.
2. **Arquitectura Modular:** El framework hace uso de programación modular, lo que quiere decir que las funciones del programa se pueden dividir en diferentes bloques libres intercambiables llamados “módulos”. Es un procedimiento de software que ofrece un desarrollo flexible y también crea una mejor cooperación entre ellos para recibir actualizaciones.
3. **Gran comunidad de desarrolladores:** Además de estar respaldado por el gigante tecnológico Facebook, *React Native* es el marco de trabajo multiplataforma más utilizado con una tasa de implementación del 42% [27]. Cuenta con más de 90k estrellas en *Github* y es un marco de trabajo muy utilizado. Una plataforma de código abierto de fácil acceso da la bienvenida a las contribuciones al marco de cada desarrollador, lo que ofrece una ventaja completa a los desarrolladores.
4. **Tiempo de recarga y depuración:** Una de sus características más atractivas es su tiempo de recarga. Incluso si la aplicación está creciendo, solo necesita 1 segundo para guardar un archivo y ver los cambios, lo cual ayuda a mantener la aplicación en ejecución mientras se agregan nuevas versiones de los archivos editados sin necesidad de recompilar de nuevo. Además, el framework cuenta con un buen sistema de depuración integrado con la consola de desarrollador de Chrome.

- **Symfony**

Symfony es un *framework* basado en el lenguaje de programación *PHP* que se utiliza para el desarrollo de aplicaciones web y es de los más utilizados en el entorno de desarrolladores de aplicaciones. Es una herramienta que facilita la creación de aplicaciones de *PHP* y *APIs* para controlar la lógica del *backend*. Este servicio consiste en que un cliente, a través de un navegador o aplicación, hace una petición, esa petición es procesada por el código servidor, ese código servidor prepara una respuesta y se la devuelve al cliente, a través del navegador. *Symfony* actúa como dicho código servidor, y ayuda a procesar y preparar las respuestas HTTP.

Las ventajas que *Symfony* presenta como framework son las siguientes:

1. **Emplea un patrón de Modelo-Vista-Controlador (MVC):** *Symfony* se basa en utilizar un patrón MVC. Éste es un patrón de arquitectura de software que separa los datos y la lógica del servicio en diferentes módulos. A los módulos de datos se les denomina modelos. Aquellos en los que se maneja la lógica, es decir, los que actúan de intermediario entre los datos y las peticiones se les llama controladores. La vista es aquella que presenta la información manejada por el controlador al usuario. En nuestro caso, como el *framework* va a ser utilizado para dar un servicio *backend*, no haremos uso de vista directamente, sino que será la aplicación *frontend* la que muestre los datos.
2. **Escalable:** Dado a la estructura que ofrece el patrón MVC, el framework ofrece un modelo sencillo de programar y fácil de escalar, separando los diferentes sistemas; rutas, control de frontales, el sistema de modelos, etc., de manera que pueda crecer de manera poco confusa y con un fácil mantenimiento futuro.
3. **Código libre:** *Symfony* presenta un modelo de código libre, u *open source*, lo que implica que el código fuente está disponible para su posible modificación y redistribución. Esto facilita la creación de librerías y módulos por parte de la gran comunidad de usuarios para el libre uso, lo que añade nuevas funcionalidades y da facilidad de uso al *framework*.

5. Normativa y legislación

5.1 Licencias

Las licencias de *software* son contratos entre el autor de un programa informático, ya sea organización o desarrollador particular, y los usuarios de este. En ellas se establecen los términos, condiciones y cláusulas que se deben cumplir para poder usar ese programa y cada usuario que se descarga, instala, copia o lo utiliza debe aceptar esas condiciones. Existen varios tipos de licencia que establecen los derechos que puede llegar a tener el usuario y lo que puede hacer con el *software*.

Las dos grandes categorías de licencias establecidas son el *software* libre y el *software* propietario. El software libre carece de *copyright*. No tiene por qué ser gratuito, pero el usuario tiene libertad sobre ese programa para usarlo, copiarlo, modificarlo y distribuirlo a su gusto. Eso permite a los millones de desarrolladores del mundo ampliar sus posibilidades y expandir saber y conocimiento. Por otro lado, el *software* de propietario es aquel que su creador o dueño legal limita o prohíbe la copia, redistribución y modificación sin su permiso o sin un pago previo. [28]

En este proyecto se ha hecho uso de licencias entre ambas categorías. Las principales han sido:

5.1.1 Licencia MIT

La licencia MIT es un *software* de licencia gratuito que otorga al usuario final del *software* derechos tales como copiar, modificar, fusionar, distribuir, etc. Su nombre deriva de las siglas en inglés del Instituto Tecnológico de Massachusetts (*Massachusetts Institute of Technology*). Es de las más utilizadas en el mundo del *software* libre. Las herramientas *React Native*, *NodeJs* o *Symfony*, empleadas en este proyecto, utilizan este tipo de licencia. [29]

5.1.2 Licencia Pública General de GNU

También conocida como GPL por sus siglas en inglés (*General Public Licence*) es probablemente la licencia de software libre más utilizada en el mundo a día de hoy. Creada por GNU y mantenida por la Fundación por el código libre (*Free Software Foundation*) mantiene la libertad del usuario de usar, estudiar, compartir (copiar) y modificar el *software*. Además, declarar un programa con una licencia de este tipo previene que se restrinjan esas libertades a nuevos usuarios cada vez que la obra es distribuida, modificada o ampliada [29]. Las herramientas que han sido utilizadas en este proyecto que siguen este tipo de licencia son por ejemplo XAAMP, Git o MySQL.

5.1.3 Microsoft 365 para estudiantes

Gracias a ser miembro de la comunidad universitaria y disponer de un correo institucional creado por la ULPGC, dispongo de una licencia de Microsoft 365 para estudiantes y miembros de la comunidad educativa, lo que me da acceso a toda la suite de productos de Microsoft *Office*. En este proyecto se han utilizado bajo esta licencia los servicios de Microsoft *Word* y *Power Point* para la redacción del informe y la presentación, además del acceso al correo institucional para el flujo de comunicaciones entre los tutores y alumno.

5.1.4 Licencias de imágenes

Las imágenes empleadas durante el uso de este proyecto que pertenecen a fuentes externas se hayan abaladas por la ley vigente de propiedad intelectual, por lo que presentan derechos de *copyright*. La LPI establece que la puesta a disposición del público por terceros de cualquier imagen, obra fotográfica o mera fotografía, divulgada periódicamente o en sitio web, estará sujeta a la autorización de su autor. Sin embargo, esta ley establece dos supuestos donde se autoriza el uso de las imágenes sin que medie la autorización de su autor:

1. **Con fines docentes o de investigación:** siempre y cuando se refieran a obras ya divulgadas y se establezca su inclusión a través de citas, comentarios o un juicio crítico.
2. **Parodia:** no se requerirá el consentimiento del autor si la transformación de la obra no crea un riesgo de confusión ni daño a la obra original.

En el caso de este informe, el uso de imágenes se resguarda dentro del ámbito de fines docentes o de investigación, por lo que se debe indicar en éstas la fuente y el nombre del autor de la obra.

5.2 Legislación respecto al tratamiento de datos

Dentro del territorio nacional se aplica la Ley Orgánica 3/2018, de 5 de diciembre, de Protección de Datos Personales y garantía de los derechos digitales, más conocida comúnmente como la **Ley Orgánica de Protección de Datos**, o LOPD, que tiene como objetivo adaptar la ley española del tratamiento de datos personales al marco legislativo europeo. Esta normativa se adapta a nivel de la Unión Europea, por lo que se deben acoger a ella todas las empresas de la Unión Europea, o que tengan negocios en ella. La Agencia Española de Protección de Datos, de ámbito estatal y creada en 1992, es la encargada de velar por el cumplimiento de esta Ley. [30]

La aplicación del proyecto recogerá datos de carácter personal de los usuarios, como son el nombre y apellidos o el correo electrónico del usuario.

De acuerdo con la ley, se garantiza el correcto uso de dichos datos, que no serán utilizados para fines que no sean estrictamente necesarios para el correcto funcionamiento de la aplicación. En ningún momento se pedirán o almacenarán datos adicionales del usuario.

6. Análisis

En este capítulo se estudiarán los requisitos que debe tener la aplicación, identificando a los diferentes actores y los casos de uso establecidos.

6.1 Actores

La aplicación cuenta con dos actores principales: **usuario registrado** y **usuario no registrado**. A continuación, se desarrollarán los diferentes actores que van a darle uso al sistema de la aplicación:

6.1.1 Invitado

Cuando un usuario accede a la aplicación y no se encuentra dado de alta, podrá acceder únicamente a funcionalidades limitadas debido a su rol, siendo éstas unas características generales. Este actor podrá:

- Ver un listado de las recetas de los usuarios, con información general.
- Acceder a las recetas de los usuarios, donde se podrá ver información más detallada.
- Búsqueda sencilla de entre el listado de recetas general.
- Compartir receta por medios externos.
- Votar una receta.
- Darse de alta en el servicio creando una cuenta.
- Acceder como usuario registrado a partir de las credenciales de su cuenta

6.1.2 Usuario registrado

Una vez un usuario crea una cuenta nueva o accede a su cuenta, si ya posee una, desbloqueará nuevas funcionalidades propias de cada usuario. Además de las funcionalidades generales del usuario no registrado, este actor será capaz de:

- Crear una receta nueva. Para ello tendrá que proporcionar información referente a ésta.
- Ver un listado de sus propias recetas creadas.
- Editar sus recetas.
- Eliminar sus recetas.
- Modificar la información de su cuenta.
- Dejar comentarios en las recetas de otros usuarios
- Añadir las recetas de otros usuarios a favoritos.
- Ver un listado de las recetas que ha añadido a sus favoritos.
- Cerrar su sesión de la aplicación.

6.2 Casos de uso

En este apartado se van a representar los casos de uso relacionados con cada uno de los actores de la aplicación. Para ello se ha hecho uso de representación en diagramas en el Lenguaje Unificado de Modelado (UML -*Unified Modeling Language*). UML es un lenguaje de modelado visual común y semántica y sintácticamente rico para la arquitectura, el diseño y la implementación de sistemas de software complejos, tanto en estructura como en comportamiento. [31]

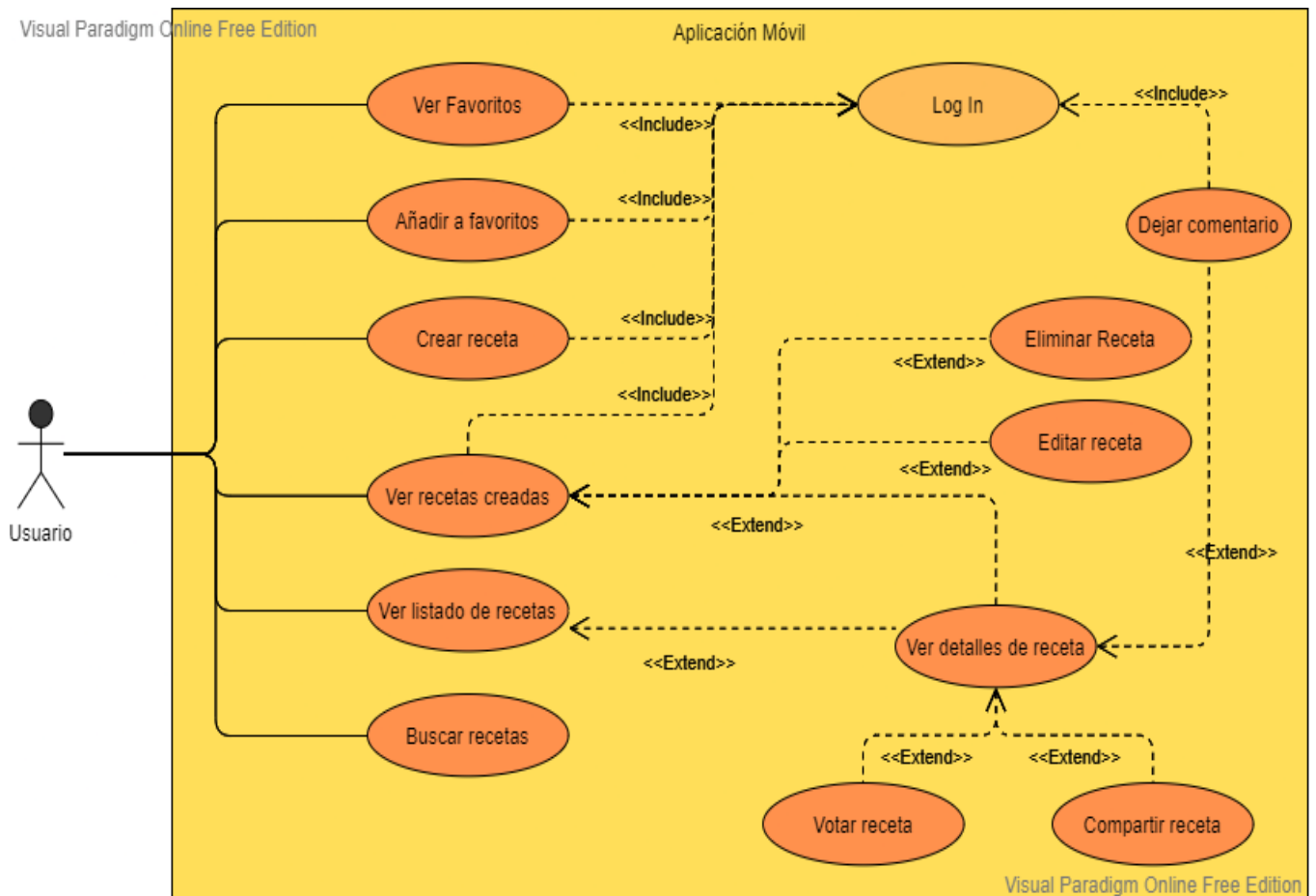


Figura 6: Diagrama de casos de uso

7. Diseño

Tras la realización del análisis de los requisitos del sistema y de los posibles casos de actuación de los usuarios de la aplicación, pasamos a la puesta a punto de un esquema funcional. El primer paso en el diseño es declarar la arquitectura que va a seguir el sistema, las ventajas que presenta ésta, y cómo se traduce su implementación a nuestra aplicación. En nuestro caso la arquitectura a seguir fue propuesta por la empresa.

7.1 Arquitectura del sistema

El concepto del sistema presenta una arquitectura **cliente-servidor**, más en concreto una arquitectura del tipo de 3 capas. Este modelo es de los más usados en la actualidad, y es en el que se basan la mayoría de los servicios, e incluso el propio internet. Cliente-servidor se traduce en una red de aplicaciones que separa las tareas entre los clientes y los servidores que viven en el mismo sistema o tienen que comunicarse a través de una red informática. El cliente-servidor se basa principalmente en enviar la solicitud a otro programa para acceder al servicio que el servidor hace accesible. Este servidor ejecuta algunos programas que comparten los recursos y distribuyen el trabajo entre los clientes. Aplicado a nuestro proyecto, la aplicación que correría en un dispositivo móvil actuaría como cliente, que será el que envíe la solicitud al servidor cada vez que requiera hacer uso de algún recurso del sistema. El servidor procesa la solicitud y devuelve una respuesta en función a los resultados.

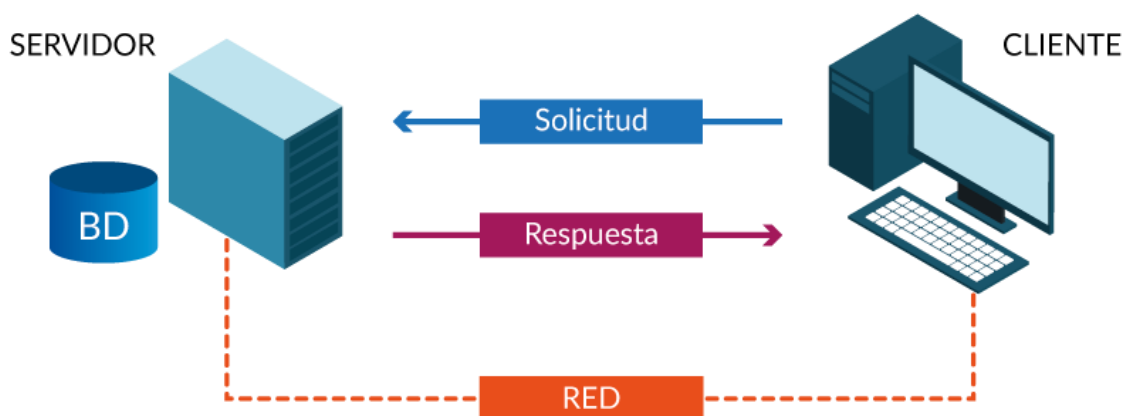


Figura 7: Ilustración de un modelo cliente-servidor

La arquitectura cliente-servidor consta de varios tipos en función del número de capas que se aplique. En nuestro caso, hemos diseñado un modelo de tres capas. Un modelo de tres capas consiste en el nivel de presentación, que es la capa de interfaz de usuario, un nivel de aplicación, que es una capa de servicio que realiza el procesamiento detallado, y un nivel de datos que consta del servidor de base de datos, que almacena la información. La arquitectura de tres niveles se puede dividir en 3 partes, la capa de presentación (o nivel de cliente), la capa de aplicación (o nivel de negocio) y la capa de base de datos (o nivel de datos). Funciona de las siguientes formas: el sistema cliente maneja la capa de Presentación; el servidor de aplicaciones se ocupa de la capa de la aplicación y el sistema del servidor supervisa la capa de la base de datos. [32] En el caso de nuestro proyecto, las capas de presentación, aplicación y base de datos son llevadas por la aplicación móvil, el servidor de la *API Rest* en *Symfony*, y el servidor SQL, respectivamente.

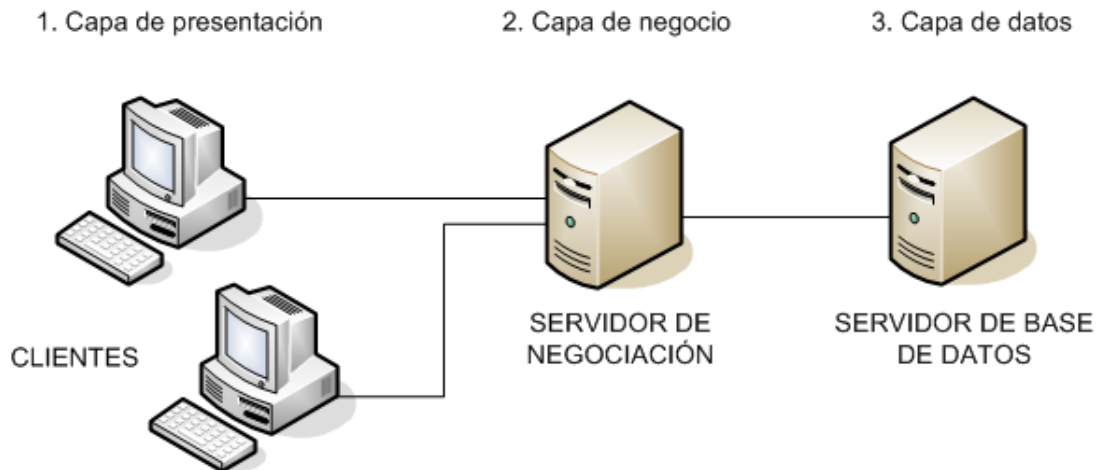


Figura 8: Modelo de cliente-servidor de 3 capas

Las ventajas que presenta este tipo de arquitectura frente a otras menos frecuentes son:

- **Administración centrada en el servidor.** Los clientes tienen poca trascendencia en el esquema y sus necesidades de administración son menores.
- **Centralización de los recursos.** Los recursos comunes a todos los usuarios se administran en el servidor. Así se evitan situaciones como la redundancia o inconsistencia de información en las bases de datos.
- **Mejora de la seguridad.** Al disponer de un mecanismo central de autenticación, las posibilidades de acceso indebido se reducen considerablemente.
- **Escalabilidad de la instalación.** Se pueden añadir o suprimir clientes sin que el funcionamiento de la red se vea afectado.

7.2 Interfaz de usuario

La interfaz gráfica de usuario, conocida también como GUI (del inglés *graphical user interface*), es la forma en que un usuario puede interactuar con un dispositivo informático. En este proyecto, la aplicación móvil actúa en gran parte como interfaz gráfica, y se centra en proporcionar un entorno visual sencillo para permitir la comunicación del cliente con el sistema.

La interfaz de usuario es de las partes más importantes y a las que más cuidado hay que poner a la hora de diseñar un producto, ya que es donde el usuario pone su atención y es la ventana que le proporciona acceso al servicio que quiere utilizar. Si le resulta complicado o poco intuitivo, se verá reflejado en una mala experiencia y dificultades para seguir el flujo de la aplicación. En nuestro proyecto es muy importante la presentación de una buena GUI, ya que estamos orientando la aplicación a un grupo de personas que ya de por sí presentan grandes dificultades en el manejo de este tipo de tecnologías.

Para el diseño de la interfaz gráfica de esta aplicación, se ha seguido el diseño proporcionado por la empresa Singular Factory, además de algunas adiciones de carácter personal, y, como ya se había mencionado en apartados anteriores, se ha utilizado *React Native* para ello. Debido a esto, la interfaz de usuario ha quedado parecida a anteriores versiones de la aplicación desarrolladas por alumnos anteriormente, aunque con pequeños cambios y mejoras. A la hora de añadir a los diseños siempre se han seguido una serie

de estándares y principios de usabilidad, con lo que se busca centrarse en el usuario y cómo interactúa en su día a día con los productos para satisfacer sus necesidades. Más adelante se atenderá a como han sido aplicados estas reglas y principios.

7.2.1 Elementos comunes

La aplicación presentará varios elementos y características comunes a todas las pantallas. La principal que se irá repitiendo será la paleta o tema de colores. Siguiendo el modelo de los diseños ofrecidos por la empresa, además de webs profesionales que ofrecen una alta variedad de temas, se ha hecho uso de una paleta de colores amarillos empleados en los elementos principales y unas tonalidades crema para los fondos y elementos secundarios, tal como se puede observar en la *figura 8*.

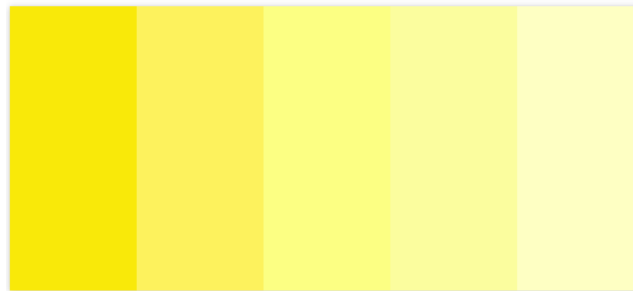


Figura 9: Paleta de colores de la aplicación

Otro elemento común a la aplicación es la barra principal. La barra de tareas es un elemento común a todas las pantallas de la aplicación, que mostrará información de la pantalla en la que se encuentre el usuario, además de diferentes botones de acción. En la principal y en las vistas de recetas como favoritos o recetas propias del usuario, por ejemplo, la barra superior mostrará un botón derecho que desplegará un cuadro de búsqueda sencilla de las recetas de los usuarios y un botón izquierdo que abrirá un menú desplegable a la izquierda de la pantalla. En otras vistas superpuestas la barra superior mostrará solo un botón de regreso en la parte izquierda.



Figura 10: Barra superior en pantalla principal

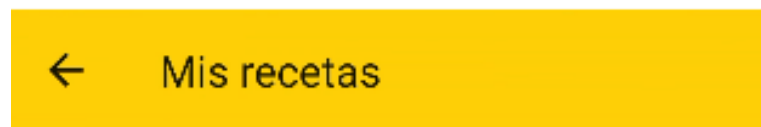


Figura 11: Barra superior en vista superpuesta

Por último, un elemento a destacar de la interfaz al que se le da uso en numerosas ocasiones, aunque no en todas las vistas, es el modal para la introducción de imágenes, ya sea por cámara o por selección de la galería del dispositivo. Ejemplos de uso de este modal se pueden encontrar a la hora de añadir foto de perfil en la cuenta del usuario o una imagen para la receta. El modal consta de un desplegable en la parte inferior de la pantalla

con dos botones y sus respectivos iconos para elegir el origen de la imagen a añadir. En función de cual se presione se realizará una acción u otra.



Figura 12: Ejemplo de modal de selección de imágenes

7.2.2 Inicio y búsqueda

Al entrar en la aplicación se mostrará la página principal, que ofrece un listado de las recetas de los usuarios de la aplicación, junto con información general de ésta. Cada elemento del listado ofrece un recuadro con la imagen de la receta, su nombre, un gráfico de opiniones en formato de 5 estrellas, tiempo de preparación y dificultad que presenta.



Figura 13: Página principal



Figura 14: Tarjeta de información de receta en el listado

A partir de esta vista principal, el usuario podrá acceder al resto de funcionalidades de la aplicación gracias al menú desplegable de la parte izquierda de la pantalla además de a una vista que ofrece una información más detallada de una receta accediendo al pulsar en ella en el listado. En función de si el usuario se halla dado de alta en el momento, el menú lateral mostrará un botón de inicio de sesión o registro o enlaces a diferentes vistas como favoritos, recetas propias del usuario o información del perfil.



Figura 16: Menú lateral de usuario invitado

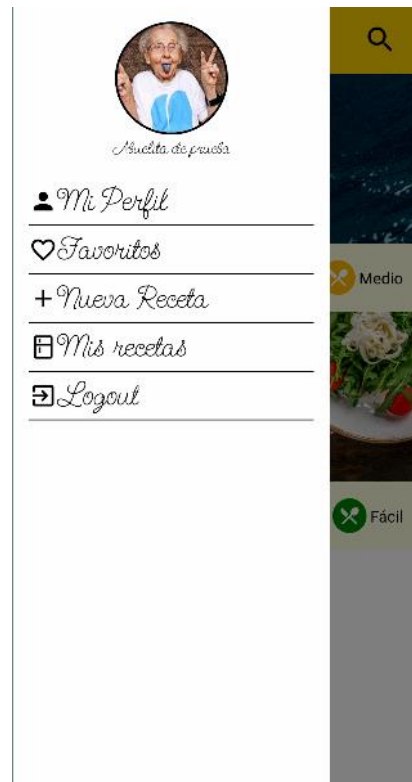


Figura 15: Menú lateral de usuario dado de alta

La función de búsqueda que se comentaba en el apartado anterior podrá realizar una búsqueda sencilla por palabra clave en el contenido del listado.



Figura 17: Ejemplo de búsqueda

7.2.3 Vista detallada de una receta

Una vez que el usuario haga *click* en una tarjeta de receta del listado general de la página principal, la aplicación mostrará una nueva vista con la información detallada de esta receta. Aquí además de la información simplificada que ya se mostraba en la página principal se podrán ver datos como la descripción de la receta, el nombre y la foto de perfil del autor, los ingredientes, los pasos, cada uno con su foto, título y descripción, y los comentarios.

Además, un usuario, tanto registrado como invitado podrá realizar una serie de acciones desde esta vista. Todos los usuarios podrán dejar una reseña en formato de voto 5 estrellas, además de poder compartir la receta en algún medio externo a su elección. Los usuarios registrados además tendrán la capacidad de añadir o quitar la receta de sus favoritos y dejar un comentario.



Figura 19: Vista completa de una receta I



Figura 18: Vista completa de una receta II

7.2.4 Vistas de autenticación

Desde la página principal, un usuario no dado de alta en el servicio podrá acceder a las opciones de iniciar sesión en su cuenta o registrarse si no posee ninguna. En la vista del registro, el usuario introducirá datos que serán utilizados luego en su cuenta de usuario en los campos destinados a ello. Los campos necesarios serán el nombre y los apellidos, el correo electrónico a utilizar, y contraseña. El campo de contraseña tendrá además una contraparte de confirmar contraseña, en el que para asegurar que el usuario lo haya introducido correctamente y evitar tener que realizar cambios, deberá repetirla correctamente y coincidir con el anterior. También, de manera opcional, podrá añadir una foto de perfil.

En el caso de que alguna de la información introducida sea incorrecta y no cumpla unas ciertas condiciones o haya habido algún error a la hora de obtener la respuesta del servidor, saltará un cuadro de diálogo mostrando al usuario los errores.

← *Registrarse*

Bienvenido! Proporcionanos los siguientes datos para darte de alta

 **Añadir foto de Perfil**

Nombre y apellidos

Email

Contraseña

Repetir contraseña

Crear cuenta

Figura 21: Pantalla de registro

← *Registrarse*

Bienvenido! Proporcionanos los siguientes datos para darte de alta

 **Añadir foto de Perfil**

Error

La confirmación de contraseña no es correcta

OK

.....

.....

Crear cuenta

Figura 20: Ejemplo de diálogo de error

Una vez que un usuario ya se haya dado de alta en el servicio haciendo uso del registro con anterioridad, podrá acceder a su cuenta a través pantalla de inicio de sesión. Esta vista plantea un formulario con dos campos en los que introducir las credenciales necesarias, el correo electrónico y contraseña. Esta vista también ofrece diálogos en caso de errores en el proceso de inicio de sesión para conocimiento del usuario.

←

Iniciar sesión



Email

Contraseña

Iniciar sesión

Figura 22: Inicio de sesión

7.2.5 Creación y edición de recetas

Las vistas de creación y edición de recetas comparten la misma estructura y flujo. Se trata de una serie de pantallas con partes del formulario de creación o edición de la receta para así evitar la saturación del usuario al mostrar muchos campos. De acuerdo con la ley de *Hick*, una ley establecida en el campo de la psicología, el tiempo que una persona tarda en tomar una decisión es proporcional a la ‘entropía de decisión’ es decir, cantidad y complejidad de alternativas que existen [33]. De esta manera, evitamos que el usuario se vea sobrepasado al encontrarse de repente con un gran número de opciones o demasiado complejo, lo que podía influir en que no acabe realizando la acción. Como se puede observar en las figuras 21, 22 y 23, los campos del formulario han sido divididos en acciones más simples y cortas, de manera que esta funcionalidad se vuelve más atractiva.

Las diferentes subformularios presentes en esta funcionalidad agrupan campos de manera organizada. En la primera pantalla, el usuario proporciona información general de la receta. Esto es: nombre, una foto de manera opcional, la descripción, dificultad y el tiempo aproximado de preparación. Seguidamente, la siguiente pantalla añade el número de personas para las que está pensada la receta y una lista de los ingredientes. La última vista es un listado de los pasos de la receta, con la que el usuario podrá interactuar de manera que pueda editar o borrar un paso y reordenar los pasos con un gesto de arrastre. Para la creación de un paso, existe otra vista separada con un formulario sencillo del paso.

The screenshot shows the 'Nueva receta' (New recipe) screen. At the top, there is a yellow header with a back arrow and the text 'Nueva receta'. Below the header, there is a blue button labeled 'SELECCIONAR IMAGEN'. Underneath, the section 'Entradilla de la receta' (Recipe introduction) is shown, with a subtext '(es una información general de la receta aprox. 5 o 6 líneas)'. A text input field contains the text 'Es un huevo frito'. Below this, the 'Tiempo aproximado de la receta' (Approximate recipe time) is displayed as '40min' with a slider control. A dropdown menu shows 'Medio' (Medium) as the selected difficulty level. At the bottom, there are two yellow buttons: 'Siguiente' (Next) and 'Cancelar' (Cancel).

Figura 24: Primer paso de creación de una receta

The screenshot shows the 'Nueva receta' (New recipe) screen, specifically the 'Ingredientes' (Ingredients) section. The header is yellow with a back arrow and the text 'Nueva receta'. Below the header, the section 'Ingredientes' is displayed in a cursive font. Underneath, there is a subtext 'Ingredientes para ...' followed by a minus sign, '4 Personas', and a plus sign. Below this, the 'Listado de ingredientes' (Ingredient list) is shown, with a subtext '(Pulsa cualquier ingrediente para editarlo)'. The list contains two items: 'Huevo' (Egg) and 'Aceite' (Oil), each with a red 'X' icon to its right. At the bottom of the list, there is a text input field labeled 'Añadir ingrediente' (Add ingredient) with a plus sign to its right. At the bottom of the screen, there are two yellow buttons: 'Siguiente' (Next) and 'Atras' (Back).

Figura 23: Segundo paso de creación de una receta



Figura 25: Tercer paso de creación de una receta

7.2.6 Creación y edición del paso de una receta

Tal como se mencionó en el apartado anterior, en una de las pantallas de creación y edición de la receta, se podrán añadir pasos de elaboración al listado de la receta. Para esto se hace uso de una nueva vista dedicada exclusivamente a completar la información de un paso. Esta vista plantea un formulario sencillo al que se le proporciona el título del paso, una imagen de forma opcional y la descripción de éste.

Figura 26: Vista de creación de un paso de elaboración

7.2.7 Mis recetas y favoritos

Desde la pantalla principal de la aplicación, el usuario podrá acceder a partir del menú desplegable a dos pantallas muy parecidas: las recetas propias creadas por el usuario anteriormente o las recetas que ha añadido a favoritos. Estas vistas están basadas en la página principal, es decir, siguen un modelo de diseño en el que se muestra las recetas de esa categoría en tarjetas con la información general de éstas.

La única diferencia notable entre estas y la principal son los datos por mostrar. En el caso de la vista de las recetas del usuario, sin embargo, la tarjeta ofrece también dos opciones adicionales: la edición y el borrado de una receta. Si el usuario presiona en el icono de editar dicha receta, navegará a la pantalla ya explicada anteriormente de editar y crear receta, con los datos de ésta en los campos respectivos. Por otro lado, si presiona el botón de eliminar una receta, saltará un diálogo para obtener una segunda confirmación por parte del usuario antes de borrar la receta.



Figura 27: Vista de recetas de usuario

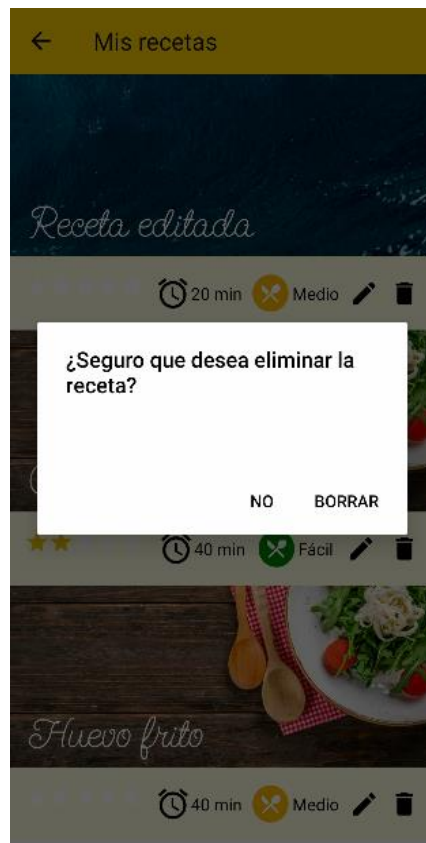


Figura 28: Diálogo de confirmación para eliminar una receta

7.2.8 Principios del software utilizados

A lo largo del diseño de la interfaz de usuario de la aplicación se ha seguido el diseño general planteado por la empresa además de adiciones propias. Para esto último se ha hecho uso de varias leyes y principios establecidos en el campo de los ingenieros especializados en el planteamiento de interfaces. A continuación, se expondrán algunas de las máximas que se han empleado para este diseño [34]:

- Coherencia: la coherencia consigue algo muy importante; que toda la aplicación se comporte siempre de manera predecible, tanto en el diseño y ubicación de sus elementos como en su funcionamiento.
- Diálogo: es importante comunicarse con el usuario a través de mensajes de texto, ya sean de error, alerta o confirmación. Deben estar correctamente redactados, ser precisos y dar la oportunidad de cancelar o deshacer una acción. En el caso de esta aplicación, se han empleado muchos diálogos de confirmación o muestra de errores, como por ejemplo para el borrado de recetas o en las validaciones de formularios.
- Control: el usuario debe tener siempre el mando y una sensación de que controla el proceso, es decir, debe poder modificar sus datos personales, cambiar la información de sus recetas, etc. Es fundamental que tenga la certeza de que puede utilizar la aplicación sin correr peligros ni riesgos.
- Proximidad: las funciones principales deben aparecer inmediatamente, mientras que otras más avanzadas pueden situarse en un lugar menos visible para los

nuevos usuarios. Además, los usuarios han de tener siempre a mano las funcionalidades más utilizadas. Por ejemplo, la búsqueda y la vista de las recetas es lo primero con lo que se encuentra el usuario al entrar, además de tener en un segundo grado de atención las funcionalidades propias de usuario registrado.

- Familiaridad: es importante que la interfaz no exija una formación específica y pueda utilizarse directamente. Esto se consigue basándose en conocimientos previos, conceptos e imágenes de la vida real del usuario. Esto se traduce en emplear las convenciones habituales de las aplicaciones informáticas en general, ya que el usuario aprovecha sus conocimientos previos en esta área.
- Sencillez: una interfaz llena de funciones avanzadas sólo consigue distraer a los usuarios. Los controles, los elementos y sus relaciones deben tener un significado obvio y comprensible, de tal modo que se entiendan de un solo vistazo.
- Atractivo: la interfaz debe ser visualmente atractiva y utilizar colores fríos o cálidos que estén en consonancia con el objetivo que se desea alcanzar o las sensaciones que se quieren transmitir. EL color amarillo está demostrado que es un captador de atención e induce a comer, además de representar la felicidad, energía, abundancia y riqueza.

7.3 Flujo de la aplicación

A continuación, se expondrá un diagrama simplificado para mostrar una visión general y el flujo planeado que seguiría un usuario durante su experiencia en la aplicación. Se pueden reconocer las diferentes vistas desarrolladas con anterioridad en el informe y la relación que ofrecen entre ellas.

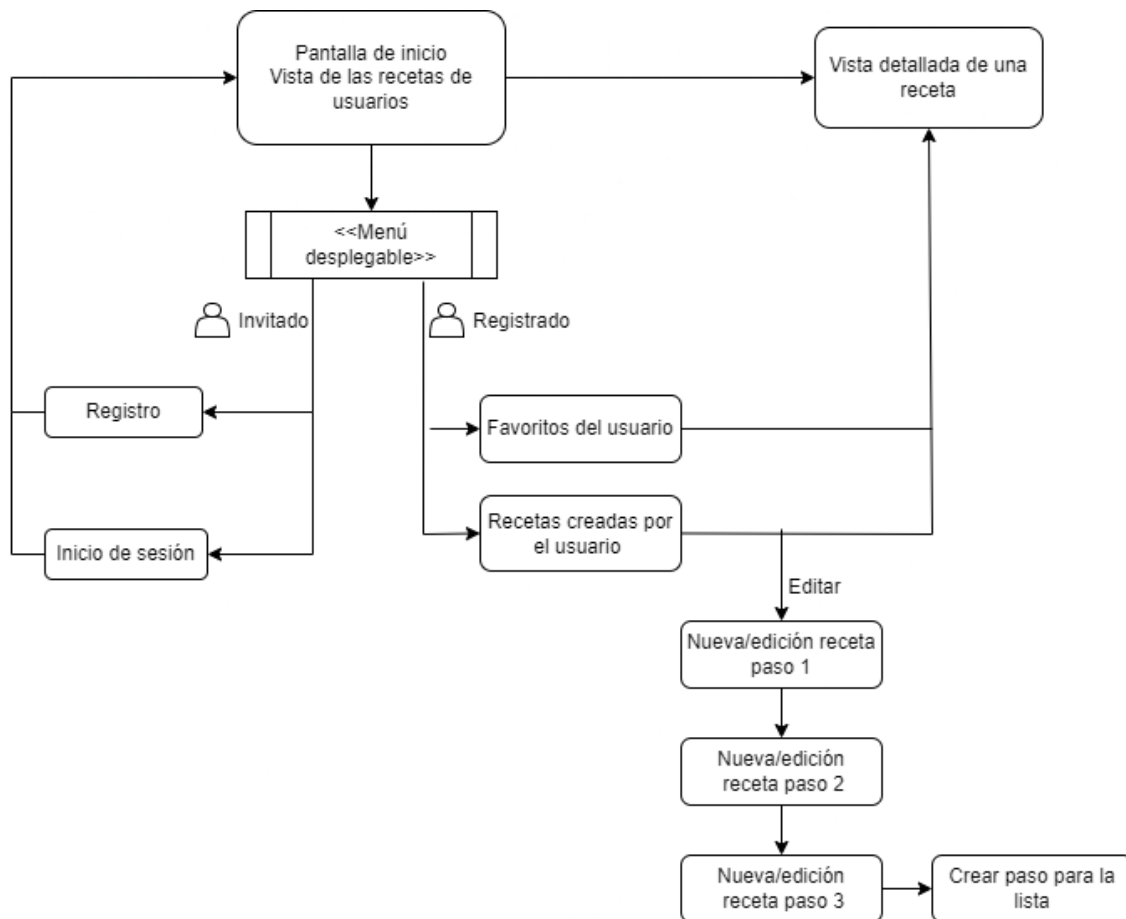


Figura 29: Visión general del flujo de la aplicación

7.4 Diseño del *Backend*

Para el diseño del *backend* de la aplicación se han estudiado los diferentes datos que van a ser necesarios almacenar para el correcto funcionamiento de la aplicación, además de sus tipos y las diferentes maneras de agruparlos. Después del análisis de la información necesaria, se ha intentado aplicar a un diseño de la base de datos, en el que se plantean las diferentes entidades y las relaciones entre ellas, además de los atributos correspondientes de cada una.

Tras este proceso, se ha acabado con un modelo como el de la Figura 30. Aquí se puede observar las entidades principales, que serán las representaciones de objetos o conceptos del mundo real. En esta categoría tenemos:

- Receta
- Usuario
- Paso de la receta
- Comentarios

Por convención, los nombres de las entidades en la base de datos serán escritas en su equivalente en inglés. Como se puede observar, cada una de estas entidades está

relacionada de algún modo a otra de entre ellas. Algunas presentan relaciones uno a muchos, como por ejemplo la relación entre Receta y Usuario. Esto quiere decir que cada receta solo puede pertenecer a un usuario, pero por el contrario cada usuario puede tener varias recetas. A la hora de desarrollar este tipo de relaciones, se presentará un atributo en la tabla de la relación a uno que apunte al identificador de ésta. Siguiendo el ejemplo anterior, la tabla Receta tendría un atributo *user_id* como clave foránea. Por otro lado, también existen en el modelo las relaciones muchos a muchos, que presentarán algo más de complejidad conceptual a la hora del desarrollo. Poniendo de ejemplo el modelo, para representar los favoritos, un usuario podrá tener muchas recetas en favoritos, y cada receta puede ser favorita a su vez de muchos usuarios. Para ello se crearía una tabla relacional favoritos o *user_recipes*, que tendrá dos claves foráneas que serán las identificadoras de cada entidad relacionada. De esta manera un mismo usuario puede estar relacionado con varias recetas favoritas y a la inversa.

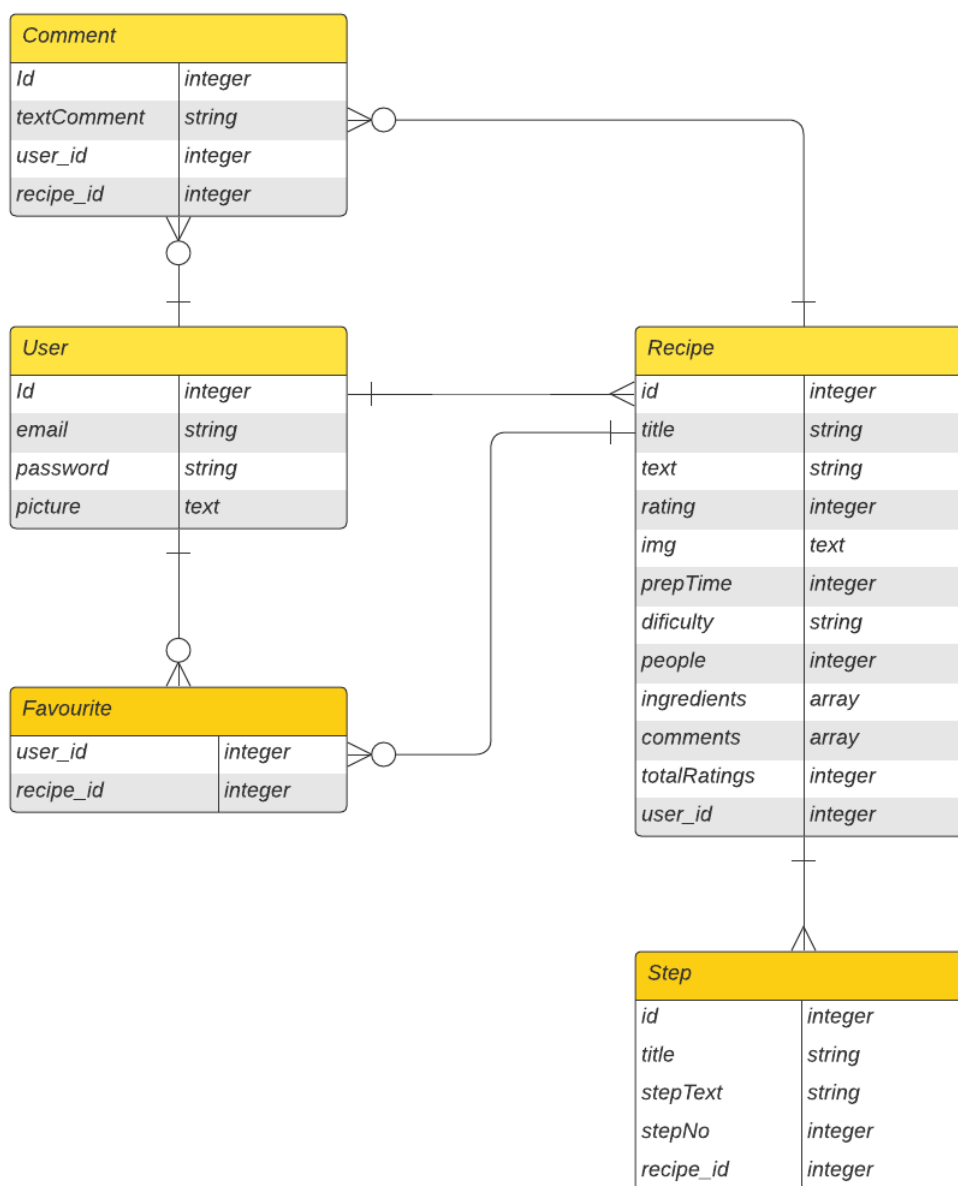


Figura 30: Diagrama de base de datos

8. Desarrollo

En el siguiente capítulo se expondrán en detalle los diferentes aspectos del desarrollo de las funcionalidades de la aplicación en sus diferentes capas, desde su estructura hasta recursos y librerías utilizadas.

8.1 Desarrollo del *backend*

Como requisito de la empresa y mejora frente al anterior prototipo de la aplicación, el lado del servidor y controlador de la aplicación ha sido desarrollado en PHP con el framework *Symfony*, ya mencionado anteriormente, de código abierto y especializado en diseño web y desarrollo de APIs.

El primer paso tras la instalación del *framework* es el de crear el proyecto. Para este proceso se ha hecho uso de la documentación oficial de *Symfony* [35]. Dicha documentación nos indica emplear el siguiente comando visualizado en la figura 31 en el directorio en el que se desee que se encuentren los archivos:

```
# run this if you are building a microservice, console application or API
$ symfony new my_project_directory --version=6.0.*
```

Figura 31: Comando de generación del proyecto de *Symfony*

Aquí se sustituirá *my_project_directory* por el nombre a utilizar, en nuestro caso **RecetasAPI**. Como vemos que comenta la documentación, este comando tiene como objetivo la generación de servicios orientados a aplicaciones, microservicios o APIs, lo que se aplica a nuestro caso. La diferencia entre este y otros comandos planteados en la documentación es que los paquetes instalados por defecto serán mucho menores, al ignorar herramientas que serán innecesarias en proyectos de esta envergadura, lo que reduce el espacio empleado, posibles brechas de seguridad y problemas de dependencias futuras.

8.1.1 Usando el modelo RESTful

Como se ha comentado, el comando genera una estructura de documentos para una API. Para los requisitos de este proyecto, el *backend* se ha desarrollado basado en los principios y un modelo de API *Rest*. Una API *Rest*, es una interfaz de programación de aplicaciones que se ajusta a los límites de la arquitectura REST y es ideal para el desarrollo de aplicaciones para dispositivos móviles. Esto se traduce a que utiliza un estilo de arquitectura que usa el protocolo HTTP en las peticiones para poder acceder y usar datos y que se adhiere a los principios de diseño REST. El nombre proviene de sus siglas en inglés (*representational state transfer* - transferencia de estado representacional).

Para que una API se considere de RESTful, debe cumplir los siguientes criterios [36]:

- Arquitectura cliente-servidor compuesta de clientes, servidores y recursos, con la gestión de solicitudes a través de HTTP.
- Comunicación entre el cliente y el servidor sin estado, lo cual implica que no se almacena la información del cliente entre las solicitudes de GET y que cada una de ellas es independiente y está desconectada del resto.
- Datos que pueden almacenarse en caché y optimizan las interacciones entre el cliente y el servidor.

- Una interfaz uniforme entre los elementos, para que la información se transfiera de forma estandarizada.
- Un sistema en capas que organiza en jerarquías invisibles para el cliente cada uno de los servidores (los encargados de la seguridad, del equilibrio de carga, etc.) que participan en la recuperación de la información solicitada.

Además, durante el desarrollo de la API del proyecto, se han seguido los siguientes estándares y buenas prácticas [37]:

- Devolver correctamente códigos de estado HTTP. Los más significativos en el desarrollo de la aplicación han sido:
 - 2xx Respuesta exitosa
 - 4xx Error en el cliente
 - 5xx Error en el servidor
- Correcto empleo de los diferentes métodos estandarizados HTTP:
 - GET: transfiere una representación actual de los datos del recurso de destino al cliente.
 - POST: realiza un procesamiento específico de recursos en la carga útil de la solicitud. Se emplea sobre todo para la creación de recursos.
 - PUT: Edita las representaciones actuales del recurso de destino con la información proporcionada en la solicitud.
 - DELETE: Elimina todas las representaciones actuales del recurso de destino.
- Devolver un recurso al realizar una llamada PUT o POST para evitar nuevas llamadas a la API.
- Incluir token de sesión en encabezados.
- Usar el formato Json siempre que sea posible.

8.1.2 Estructura general del proyecto

El proyecto presenta una estructura bastante estándar en donde se almacenan los archivos en diferentes directorios en función de su funcionalidad y servicio que ofrecen. A continuación, se analiza la ordenación de los directorios más importantes.

Dentro del directorio de **/config** se guarda toda la información del proyecto y la configuración tanto propia como de los diferentes *bundles* o librerías añadidas al proyecto. También aquí se podrá encontrar la configuración de rutas y los archivos de servicios.

El directorio **/migrations** almacenará los archivos de migración generados por *Doctrine*, que guardan las modificaciones en formato de versiones que se han realizado en la base de datos, para revertir de manera fácil entre diferentes esquemas.

Todo lo que es el código de la api se encontrará dentro del directorio **/src**, donde a su vez se organizan en función de funcionalidades:

- **/src/Controller:** directorio donde se almacenan los archivos que contienen las clases controlador. Estas clases contienen toda la lógica de la aplicación, que manejan las peticiones y deciden qué responder en función del contexto y la entidad.

- **/src/Entity:** directorio de entidades generado por *Doctrine* que representa las diferentes tablas de la base de datos mapeadas en forma de clases de entidades, con sus atributos como variables. Facilita mucho el manejo de entidades de la base de datos, con sus métodos *get* y *set* (obtener y modificar) y sus relaciones.
- **/src/Repository:** directorio de repositorios de las entidades, que ofrece llamadas útiles a la base de datos para recuperar objetos, como son las búsquedas.
- **/src/Form/Type:** directorio que se ha utilizado para los diferentes formularios generados por el *form bundle*, una librería que ayuda a simplificar las entradas al controlador de las peticiones de manera que se puedan obtener en una entidad y validar de manera sencilla.

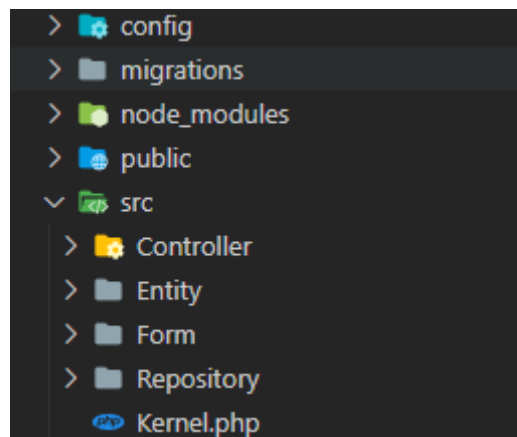


Figura 32: Estructura general del proyecto

8.1.3 Implementación de *Doctrine*

Continuando con el desarrollo del *backend*, se realiza la implementación de la base de datos con la api. Ya que la principal funcionalidad de una api es gestionar las peticiones y respuestas del usuario en función de los datos almacenados en la base de datos, este es un proceso muy importante en el desarrollo funcional del *backend*. *Symfony* por suerte ofrece un servicio llamado *Doctrine* que simplifica a los desarrolladores la creación de entidades y conexiones con la base de datos. A continuación, se procede a ofrecer contexto y explicación de sus funcionalidades, además de las ventajas que ofrece:

Doctrine Query Language es un mapeador relacional de objetos, u ORM (por sus siglas en inglés *Object relational mapper*) para PHP que se asienta sobre una potente capa de abstracción de bases de datos. Una de sus características clave es la opción de escribir consultas a la base de datos en un dialecto SQL orientado a objetos propio. Esto proporciona a los desarrolladores una potente alternativa a SQL que mantiene la flexibilidad sin requerir una duplicación de código innecesaria. Gracias a esta herramienta, un desarrollador es capaz de generar tablas y consultas con mucha facilidad, además de tratado de entidades como si fuesen objetos de PHP, y *Doctrine* se encarga de “traducir” este dialecto a consultas SQL [38].

Para el desarrollo del proyecto se ha utilizado esta herramienta que ha servido como apoyo para la creación y manejo de las entidades que va a necesitar la aplicación. *Doctrine* genera las entidades por un comando por la consola que se mapean en ficheros dentro de una carpeta llamada *Entity*, tal como se puede observar en la figura X. En estos archivos se puede ver como trata estas entidades, donde los atributos de la tabla los transforma a

variables dentro de una clase, además de generar automáticamente métodos de obtención y modificación para cada atributo. Además, también ofrece unos archivos denominados repositorios, que ofrecen métodos como son los diferentes tipos de búsqueda, en función de atributos, de id o generales, de todo el repositorio de entidades de la base de datos. Gracias a esto se puede trabajar de manera fácil luego en los controladores manejando los datos de la base de datos.

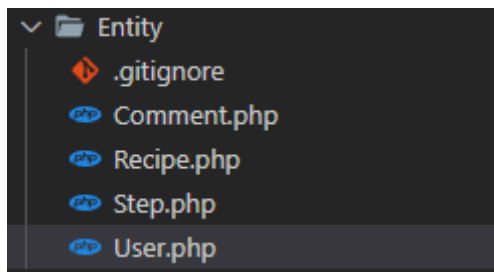


Figura 33: Entidades del proyecto generadas por Doctrine

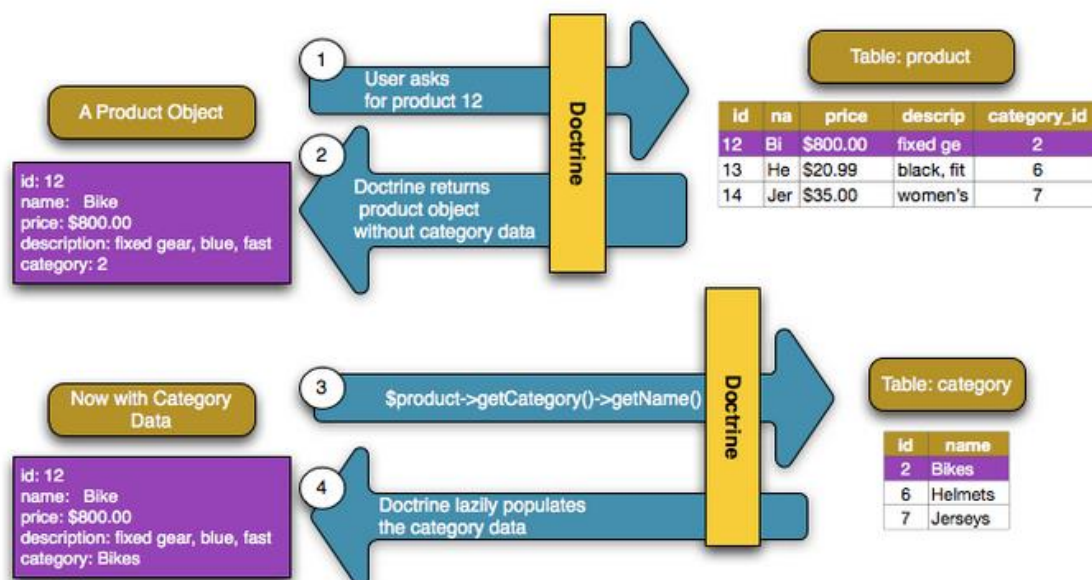


Figura 34: Diagrama de ejemplo de flujo de trabajo de Doctrine [56]

8.1.4 Autenticación y Seguridad

Una de las partes más importantes de un servicio de Api es la garantía de los datos y seguridad de la aplicación, sobre todo cuando va a existir información comprometida que solo podrá ser accesible desde usuarios autenticados. Para la implementación de estas funcionalidades se ha hecho uso de un *bundle* o librería concreta que facilita y automatiza el servicio de darse de *login* por parte de un usuario, además de otorgar muchas más funcionalidades al programa. Esta librería es muy conocida en la comunidad de desarrolladores de *Symfony*: *LexikJWTAuthenticationBundle* [39].

Gracias a esta librería se puede integrar un servicio sencillo de autenticación por *Json Web Token*, de ahí el nombre del *bundle*. Las dos funcionalidades principales que este paquete añade a la api son:

- Servicio de autenticación de usuario: una vez que un usuario establezca sus credenciales en la aplicación, estos serán enviados a `/login_check`, una ruta configurada por el *bundle* que comprobará automáticamente si el usuario se encuentra en la base de datos y si la contraseña cifrada es correcta. Una vez haya gestionado la librería, devolverá en caso de acierto una respuesta con la información del usuario, además de un token de autenticación que será almacenado en el dispositivo del cliente de la aplicación y que será enviado posteriormente con las peticiones en la cabecera, junto con la etiqueta de *Authenticate*.
- *Firewall*: este servicio ofrece una manera de evitar que usuarios no autenticados accedan a rutas y recursos de la api o base de datos sin permiso explícito. El firewall se configura de manera que ciertas rutas estarán boqueadas a no ser que se pase en la cabecera de la petición un token ya proporcionado antes por parte del servicio de autenticación del usuario. De esta manera, evitamos que cualquiera pueda realizar una petición por HTTP y recoger datos comprometidos de la base de datos. Las peticiones que no tengan la credencial en la cabecera recibirán la respuesta con un código de estado 401 *Unauthorized*.

Una vez descargado e instalado, junto con la librería de OpenSSL, para la gestión de claves públicas y privadas que se utilizarán a la hora de generar los tokens, se generará en el árbol de directorios del proyecto varios archivos que servirán para la configuración:

- **`/config/jwt`**: aquí se almacenarán las claves pública y privada en formato .pem generadas por OpenSSL y que servirán para el servicio de autenticación.
- **`/config/packages/lexik_jwt_authentication.yaml`**: en este archivo se almacena principalmente configuración relacionada con el token que se enviará al usuario para manejar la autenticación de acceso a rutas, además de configuración más general del *bundle*. Aquí se establecen parámetros como la clave pública a utilizar, el tiempo de vida del token o qué campo de la tabla de usuarios será el utilizado para validar la autenticación.
- **`/config/packages/security.yaml`**: archivo de configuración del apartado de seguridad de la api. Aquí se especificarán temas como los *user_providers*, es decir, la entidad encargada de manejar a un usuario, en nuestro caso la entidad denominada *user*; o la configuración del firewall. El firewall es un elemento muy importante de la aplicación y aquí debe de estar muy bien configurado para evitar brechas de seguridad y accesos no debidos. También se puede configurar la ruta de *login_check*, es decir, la ruta que maneja el inicio de sesión por parte del usuario. En este archivo se le especifica a la ruta que acciones realizar en caso de acierto o fallo del inicio de sesión del usuario. Para este proyecto se ha dejado esta configuración por defecto, pero podrían añadirse nuevas funcionalidades desde este área.

```

firewalls:
  dev:
    pattern: ^/(_(profiler|wdt)|css|images|js)/
    security: false
  login:
    pattern: ^/api/login
    stateless: true
    json_login:
      check_path: /api/login_check
      success_handler: lexik_jwt_authentication.handler.authentication_success
      failure_handler: lexik_jwt_authentication.handler.authentication_failure

  api:
    pattern: ^/api
    stateless: true
    jwt: ~

```

Figura 35: Archivo security.yaml en la configuración del firewall

8.1.5 Otras librerías y herramientas configuradas

- FOSRestBundle.

Librería desarrollada por la comunidad de desarrolladores de Symfony que ha sido implementada dentro del proyecto principalmente para poder utilizar un servicio de serializador en las respuestas de los controladores. Gracias a esta librería, se pueden crear archivos personalizados de grupos de serializador, lo que ayuda a simplificar el cuerpo de los métodos de controladores. Estos archivos son creados por el usuario y especifican los campos de una entidad que queremos que devuelva la herramienta. De esta manera, al especificar el serializador en un método, recoge los campos de la respuesta cual título coincida con los escritos en el archivo correspondiente y los devuelve juntos en formato Json automáticamente. Esto ayuda a aligerar el código y a simplificar el número de campos en función de la respuesta [40].

```

App\Entity\User:
  attributes:
    id:
      groups: ['user']
    email:
      groups: ['user']
  recipes:
    groups: ['user']
  picture:
    groups: ['user']
  name:
    groups: ['user']

```

Figura 36: Ejemplo de serializer de Usuario

- Form Types.

Crear y procesar formularios HTML es difícil y repetitivo. Hay que lidiar con la representación de los campos del formulario HTML, la validación de los datos enviados,

el mapeo de los datos del formulario en objetos y mucho más. *Symfony* incluye una potente función de formularios que proporciona todas estas características y muchas más para escenarios realmente complejos, llamada *Form Types*. Gracias a esta herramienta, podemos especificar en un archivo una nueva clase que declara un objeto de tipo *Form*. A este objeto se le pasa la solicitud de formulario y automáticamente trata la petición. De esta manera se valida y devuelve un objeto del tipo de entidad de la base de datos si la petición se ajusta a las condiciones especificadas en la clase *FormType*, lo que facilita mucho la creación de nuevos objetos con muchos parámetros, sin necesidad de ir uno por uno comprobando que existe en el cuerpo de la petición los campos que se ajustan a la entidad y asignarlos a ésta [41].

```
class RecipeFormType extends AbstractType
{
    public function buildForm(FormBuilderInterface $builder, array $options): void
    {
        $builder
            ->add('title', TextType::class)
            ->add('text', TextType::class)
            ->add('rating', NumberType::class)
            ->add('img', TextType::class)
            ->add('prepTime', NumberType::class)
            ->add('difficulty', TextType::class)
            ->add('people', NumberType::class)
            ->add('ingredients', CollectionType::class, array(
                'entry_type' => TextType::class,
                'allow_add' => true,
            ));
    }
}
```

Figura 37: Form Type de las recetas

En las figuras 38 y 39 se puede observar cómo simplificaría una petición de creación de la clase receta. En la segunda de éstas se ve como el formulario automáticamente recoge los atributos de la petición y los mapea en una entidad del tipo receta.

```
$recipe->setTitle($request->get('title', null));
$recipe->setText($request->get('text', null));
$recipe->setIngredients($request->get('ingredients', null));
$recipe->setImg($request->get('img', null));
$recipe->setPeople($request->get('people', null));
$recipe->setDifficulty($request->get('difficulty', null));
$recipe->setPrepTime($request->get('prepTime', null));
```

Figura 38: Ejemplo de creación de receta sin FormType

```
$recipe = new Recipe();
$form = $this->createForm(RecipeFormType::class, $recipe);

$form->handleRequest($request);
```

Figura 39: Ejemplo de creación de receta con FormType

- Repositorios

Cuando se va a realizar una consulta en la base de datos respecto a algún tipo de objeto en particular, se utiliza lo que se conoce como su "repositorio". Un repositorio se puede plantear como una clase de PHP cuyo único trabajo es ayudar a obtener entidades de una determinada clase. Viene generado junto con *Doctrine* y facilita las operaciones entre la API y la base de datos. A parte de poder crear métodos personalizados, los repositorios nos ofrecen ya algunos por defecto que se observan en la figura 40:

```
/**
 * @method Recipe|null find($id, $lockMode = null, $lockVersion = null)
 * @method Recipe|null findOneBy(array $criteria, array $orderBy = null)
 * @method Recipe[]    findAll()
 * @method Recipe[]    findBy(array $criteria, array $orderBy = null, $limit = null, $offset = null)
 */
```

Figura 40: Métodos por defecto del repositorio de receta

- *Find*, se le proporciona un identificador y devuelve un objeto o nulo en función de si se encuentra o no.
- *FindOneBy*, parecido al anterior, devuelve el objeto o nulo si no lo encuentra, pero se le proporciona una lista de criterios y el orden (ej. ascendente o descendente).
- *FindAll*, devuelve toda la lista de objetos.
- *FindBy*, parecido a *FindOneBy* pero devuelve todos los objetos que coincidan con los criterios de búsqueda.

8.1.6 Controladores y métodos de manejo de llamadas

A continuación, se va a explicar con detalle cual es la estructura que se ha seguido para la redacción de los métodos que manejan la lógica de las llamadas a la api. Estos métodos están organizados de manera que se encuentran en el controlador relacionado con una entidad general de los datos que manejan, y en ellos se escribe el código que encapsula las acciones a realizar con dichos datos. Además, se agrupan en función al tipo de acción, como por ejemplo de obtención, editado o borrado.

En primer lugar, se ha de explicar el concepto de las anotaciones de PHP. Las anotaciones (*annotations* en inglés) son básicamente metadatos que se pueden incluir en formato de comentarios en el código fuente y también entre clases, funciones, propiedades y métodos. Deben comenzar con el prefijo *@* dondequiera que se declaren e indican algo específico. La misma anotación puede ser usada para gran variedad de propósitos, como por ejemplo para determinar qué tipo de entrada necesita ser dada en un formulario y también para propósitos de automatización [42]. Ya con anterioridad habíamos visto un ejemplo de uso de anotaciones en el uso de métodos de repositorios. Este concepto es importante ya que es lo primero que se ve que se aplica en los métodos de los controladores del proyecto.

Para seguir la explicación, se ha recogido un método sencillo desarrollado en el proyecto, como es el de eliminar receta, como ejemplo. El método está ilustrado en la figura 41.

Como se puede observar, el primer bloque de comentarios contiene dos anotaciones. La primera describe el método estándar HTTP a seguir, además de la ruta que se quiere que

tenga el método. En el caso del ejemplo, se describe el método *Delete*, ya que lo que se quiere realizar es una lógica de borrado. En otros casos, como por ejemplo pueden ser los métodos de obtención de datos, se usaría el *Get*. Luego la ruta define la dirección a la que tendría que enviar la petición el cliente para obtener el resultado de la lógica de este método. En este ejemplo la ruta asignada es `"/recipe/delete/{id}"`. Cuando en la ruta se define una palabra entre corchetes quiere decir que el siguiente método puede utilizar lo que se encuentre en esa parte de la ruta de la solicitud como variable, y utilizará de nombre la palabra utilizada. En este caso vemos que la solicitud enviará la identificación o Id de la receta a eliminar en la ruta.

En la segunda línea de anotaciones se ve como se declara el grupo de serializador que se va a aplicar a la respuesta de este método. Los serializadores como ya han sido explicados anteriormente, se utilizan para facilitar la salida de datos en formato Json del método. En este ejemplo, el serializador pertenece al grupo de recetas y se utilizará al retornar la receta que ha sido eliminada.

Por último, se observa ya lo que es el cuerpo del método, donde se introduce el código que va a ejecutar este método. Funciona como método estándar de PHP, donde se declara una función pública. En este caso la función tiene de nombre *deleteRecipe*, y que se le proporciona el repositorio, en este caso de recetas que ofrece método de buscar y borrar, además del identificador que, como se mencionó con anterioridad, se captura a través de la ruta.

```
/**
 * @Annotations\Delete(path="/recipe/delete/{id}")
 * @Annotations\View(serializerGroups={"recipe"}, serializerEnableMaxDepthChecks=true)
 */
public function deleteRecipe(int $id, RecipeRepository $recipeRepository){
    $recipe = $recipeRepository->find($id);
    $recipeRepository->remove($recipe);
    return $recipe;
}
```

Figura 41: Función de borrar receta.

8.2 Desarrollo del *FrontEnd*

El lado del *fontEnd* del proyecto, es decir, la aplicación en sí; herramienta que utiliza el usuario para visualizar e interactuar con los datos, esta desarrollada, como ya ha sido definido en la planificación, con el *framework* de *React Native*. Funciona en sinergia con el *backend* de manera que conecta a través de peticiones realizadas por el usuario y éste las procesa, ofreciendo una respuesta adaptada a la necesidad del usuario, como pueden ser modificación o acceso a datos de la base de datos en un determinado momento.

8.2.1 Estructura general de la aplicación

Gracias a la empresa para la que se ha desarrollado el proyecto, fue proporcionado al inicio de la fase de desarrollo un repositorio que presentaba un proyecto base del que partir con la instalación realizada y varias librerías instaladas. Esto resultó ser de gran utilidad, ya que aportaba una gran cantidad de módulos de gran rendimiento que ya se

encontraban configurados, además que sirvió para evitar errores de dependencias comunes que se suelen encontrar entre versiones de diferentes módulos.

A pesar de que la documentación oficial de *React Native* no ofrece ninguna recomendación respecto al tipo de estructura que deben seguir los ficheros del proyecto, si afirma que existen varias formas comunes de ordenación, como son la estructura de agrupamiento por rutas o por tipo de archivo [43]. Para este proyecto se ha utilizado la estructura de agrupamiento por tipo de archivo, donde se juntan los archivos de tipos similares juntos. Además, se ha ido un paso más allá y, por ejemplo, en el caso de los archivos de componentes, se han agrupado a su vez en carpetas para cada componente.

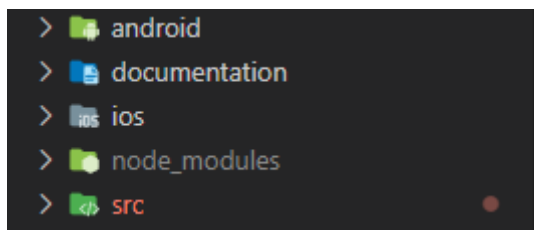


Figura 42: Estructura general del proyecto

Empezando por la estructura general del proyecto, tal y como se ve representado en la figura 42, contamos con cinco directorios principales en el primer nivel del proyecto base proporcionado por la empresa.

En la carpeta **/node_modules** es donde uno encuentra la información y configuración de los diferentes paquetes y módulos instalados que se quieren implementar en el proyecto. Cada vez que se instala un paquete nuevo desde *npm*, una de las herramientas descritas al inicio del informe que se utiliza para automáticamente instalar componentes nuevos desde *Node*, se hace bajo este directorio, además de que la información de las librerías, versión y atajos de comandos que utilizan se grabarán en el archivo **packages.json**. Este archivo se encuentra a su vez en la raíz del proyecto, fuera de estos directorios.

La carpeta **/documentation** ofrece documentación adicional del proyecto base con instrucciones sobre la configuración y la puesta a punto de ciertas funcionalidades y módulos que vienen dentro de éste.

El principal directorio sobre el que se ha trabajado es la carpeta **/src**, donde se encuentra todo el código desarrollado durante el proyecto. Este código está escrito en archivos de formato *JavaScript* y el propio *React Native* acabará transformando los componentes en componentes nativos de la plataforma *Android* o *iOS*. Los archivos de configuración y de compilación nativa de estas plataformas estará contenido dentro de los directorios respectivos **/android** y **/ios**. Dentro de la carpeta **/src** se puede observar a su vez otro repertorio de directorios. Los más interesantes a tratar son la carpeta **services**, **resources**, **routes** y **modules**:

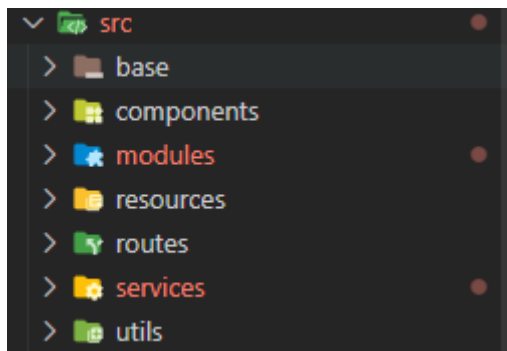


Figura 43: Estructura del directorio /src

- **/src/services** contiene los servicios utilizado por la aplicación, como principalmente los empleados para declarar las llamadas y conectar con la API en el *backend*.
- **/src/resources** es el directorio donde se almacenan los archivos que se usarán como recursos de la aplicación, como imágenes, estilos globales y archivos para la traducción.
- **/src/routes** es una carpeta muy importante dentro del proyecto, ya que aquí se encuentran los archivos **Routing.js** y **Router.js**, empleados para configurar la navegación entre las diferentes pantallas y estructura de las vistas de la aplicación.
- **/src/modules** es de los más importantes de la aplicación. Aquí se declaran los diferentes módulos y contenedores de los que se hará uso para la construcción de la interfaz de usuario junto con el archivo de estilos respectivo de cada uno. Además, en el código dentro de los archivos de los módulos aplica un grado de lógica para que el cliente pueda interactuar dinámicamente de forma sencilla con la interfaz. Tal y como se desarrolló anteriormente, este proyecto sigue una estructura de agrupación por modo en la que, además, se agrupan los diferentes módulos en subcarpetas, como se puede apreciar en la figura 44.

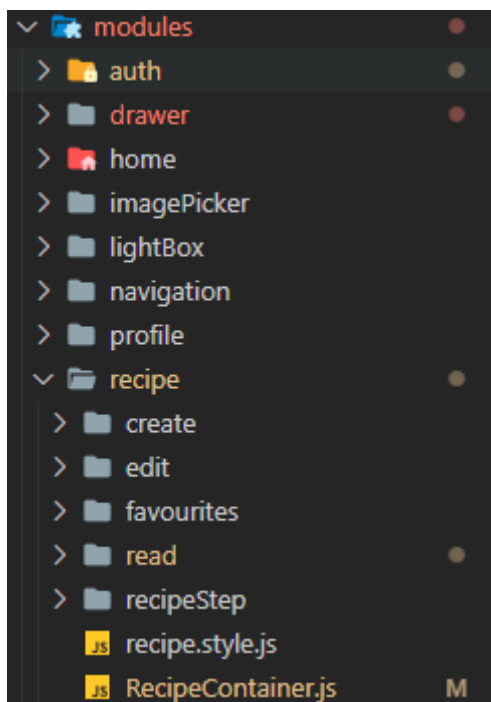


Figura 44: Estructura del directorio /src/modules

8.2.2 Módulos utilizados

En este apartado se describirán algunas de las principales librerías utilizadas en el proyecto. Tal como se mencionaba con anterioridad, todos estos módulos han sido descargados e instalados automáticamente haciendo uso de la herramienta *npm*.

- ***React Native Router Flux***

Este módulo es un derivado del famoso *React Navigation*, una librería diseñada para la creación de navegación entre vistas y ventanas de la aplicación con facilidad, que añade además una barra de navegación. Gracias a esto, se puede tener de manera centralizada en un solo archivo las distintas vistas y su configuración [44].

- ***React Redux***

Redux es un contenedor de estado predecible para aplicaciones JavaScript. Ayuda a escribir aplicaciones y poder añadir un comportamiento consistente a los datos, es decir, centralizar cierta lógica y estados entre componentes de la aplicación, de manera que componentes pueden compartir estados o funciones entre ellos de manera fácil. Es pequeño, pero tiene un gran ecosistema de complementos disponibles [45].

- ***React Native Searchbar***

Componente que añade una barra de búsqueda para *React Native*. Tiene una capacidad de búsqueda simple incorporada que se puede utilizar entregando los datos del componente y proporcionando una función *handleResults* que recibirá la copia filtrada de la matriz de datos. La función de búsqueda utiliza un algoritmo de profundidad para recorrer los datos y simplemente busca un *indexOf* la entrada del usuario dentro de todas las cadenas, números y valores booleanos en los datos [46].

- ***React Native Image Picker***

Un módulo de *React Native* que permite seleccionar una foto o vídeo de la biblioteca del dispositivo o de la cámara, además de presentar una gran cantidad de opciones para el manejo de la imagen o imágenes seleccionadas e introducidas en la aplicación [47].

- ***React Native Vector Icons***

Los iconos sirven como ayuda visual para que los usuarios naveguen por su aplicación, y la elección de la fuente (y el color de esta) establece el tono de la aplicación o marca. *Vector Icons* ofrece un extenso paquete de diferentes fuentes, como por ejemplo los *Material Icons*, mantenidos por Google, con una gran librería de iconos que mantienen variedad, sencillez y resultan sencillos de integrar en el código de la aplicación [48].

- ***i18n-js***

Módulo integrado en el proyecto base que permite la fácil integración de diferentes idiomas en la aplicación dependiendo del idioma principal del dispositivo móvil desde el que se navega. Centraliza traducciones de cada idioma en forma de objetos *JavaScript* en sus respectivos ficheros [49].

- ***React Native Star Rating***

Un componente de *React Native* para generar y mostrar clasificaciones de estrellas interactivas compatible con *iOS* y *Android*. Posee una gran variedad de opciones de

personalización además de ofrecer facilidad de integración para el desarrollador y de que el usuario obtenga una experiencia fluida y rápida al calificar [50].

- **React Native Modal**

Componente basado en la funcionalidad nativa de *React Native* de los modales que añade muchas más funcionalidades y efectos. Ofrece animaciones de entrada y salida, una API sencilla y flexible, opacidad, color y fondo personalizables, funciones para la finalización de las animaciones modales y desplazamiento dinámico [51].

- **React Native Slider**

Modulo que añade un componente “<Slider>” de *JavaScript* puro para *React Native*. Este componente de fácil inserción en el código ofrece una manera fácil de que el usuario inserte un input numérico mediante una barra deslizable. Además, ofrece una amplia gama de opciones de personalización, desde dimensiones o colores e iconos utilizables hasta la distancia entre cada “paso” [52].

- **React Native Draggable Flatlist**

Un componente basado en el componente nativo de *React* conocido como *Flatlist*. Esta librería ofrece una lista con capacidad de arrastrar y soltar con funciones y métodos para el comportamiento de los elementos al arrastrar o soltar [53].

- **React Native Splash Screen**

Componente integrado en el proyecto base. Ofrece una API de pantalla de bienvenida para *react-native* que puede ocultar y mostrar la pantalla de bienvenida de forma programática. Funciona en *iOS* y *Android*. La pantalla de bienvenida es un elemento muy común en las aplicaciones comerciales de dispositivos móviles para mostrar mientras cargan [54].

8.2.3 Navegación

Otro módulo de la aplicación desarrollado importante para el correcto funcionamiento de ésta es el módulo de navegación. Como se ha mencionado con anterioridad, esta funcionalidad se implementa con facilidad gracias a la librería de *react-native-router-flux*, que ofrece ayuda al desarrollador a la hora de programar la navegación entre vistas. Para el desarrollo de este módulo se han especificado las configuraciones en dos archivos: **Router.js** y **Routing.js**, que se encuentran en la carpeta **/src/routes**.

Empezando por el archivo **Routing.js**, en este se establecen las acciones y cierta lógica que va a seguir la navegación de la aplicación, como los identificadores de vistas o métodos de navegación. Si se observa el archivo podemos ver en primer lugar los nombres o identificadores de las diferentes pantallas de la aplicación almacenadas en constantes exportables. Estas variables se escriben aquí de manera que estén centralizadas y se puedan llamar siempre que se necesite navegar a una de ellas o establecer el nombre una vez se crean.

```

/** ***** */
/** Full Screens */
/** ***** */
export const home = 'home';
export const recipe = 'recipe';
export const userRecipes = 'userRecipes';
export const editUserRecipes = 'editUserRecipes';
export const userFavourites = 'userFavourites';
export const createNewRecipe = 'createNewRecipe';
export const createNewRecipeStep = 'createNewRecipeStep';
export const editRecipeStep = 'editRecipeStep';
export const profile = 'profile';

```

Figura 45: Identificadores de las rutas en el archivo Routing.js

Más adelante, se pueden identificar los diferentes métodos utilizados en la navegación entre pantallas. Aquí se hace uso de los identificadores para crear funciones específicas de salto a pantallas específicas. Como se puede observar en la Figura 46, el ejemplo de abrir la pantalla de la información de la receta establece la navegación hacia esa pantalla haciendo uso de la acción de navegación general al que se le establece como parámetros el identificador de *recipe* y una copia de los parámetros pasados a esta función, además de una comprobación de que el usuario no se encuentra ya en dicha vista.

```

export const openRecipes = (...params) => {
  if (!(currentScene() === recipe)) route(recipe, ...params);
};

```

Figura 46: Función personalizada de navegación a la vista de detalles de receta

Por último, este archivo establece simplificaciones de las acciones generales de navegación ofrecidas por *react-native-router-flux*, con el objetivo de reducir el número de importaciones cada vez que se quiera hacer uso de la navegación al tenerlo todo más centralizado. Las principales funciones que se establecen aquí son:

- *currentScene*: Devuelve la escena actual que está activa.
- *drawerOpen*: Si es posible, abre el menú desplegable.
- *drawerClose*: Si es posible, cierra el menú desplegable.
- *pop*: Vuelve a la escena anterior "sacando" la escena actual de la pila de navegación.
- *popTo*: Despliega la pila de navegación hasta llegar a la escena con el identificador especificado.
- *refresh*: Recarga la escena actual cargando nuevos parámetros en la escena.
- *route*: Se utiliza "automáticamente" la clave de la escena especificada por la función en el router para navegar.

```

/** ***** */
/** Methods */
/** ***** */
export const currentScene = () => Actions.currentScene;

export const drawerClose = () => Actions.drawerClose();

export const drawerOpen = () => Actions.drawerOpen();

export const pop = (params) => Actions.pop(params);

export const popTo = (key, params) => Actions.popTo(key, params);

export const refresh = (params) => {
  if (params) {
    Actions.refresh(params);
    return;
  }

  Actions.refresh({ key: Actions.currentScene });
};

export const route = (key, params) => Actions[key](params);

```

Figura 47: Acciones generales de navegación

En el archivo **Router.js** es donde se configuran las rutas en sí del proyecto, con su flujo y su jerarquía. La librería utilizada ofrece varias etiquetas que se pueden establecer en el código que ayudan a crear de forma fácil estas vistas. En la Figura 48 se puede observar las diferentes pantallas creadas y como se han empleado dichas etiquetas. En primer lugar, se describe la pantalla principal o *home*, dentro de una etiqueta `<Drawer>`. Esto quiere decir que esta pantalla será capaz de abrir el menú desplegable de la izquierda de la pantalla de la aplicación. La siguiente etiqueta que se repite en el código de las rutas es la de `<Scene>`. Así es como *react-native-router-flux* denomina a las vistas de la aplicación. Se puede apreciar los diferentes parámetros pasados a cada una de las vistas. En primer lugar, el parámetro *key* establece el identificador de la vista, que será usado para la navegación entre escenas. Este identificador se importa de las variables con los identificadores del archivo **Routing.js** explicado en el apartado anterior. Lo siguiente es el parámetro *component*, el cual establece el componente que se quiere mostrar en la escena. Por último, *navigarionBarStyle* especifica el estilo que se desea que tenga la barra de navegación, ya que las escenas de *react-native-router-flux* vienen integradas con una barra de navegación por defecto, a no ser que se le añada a la escena la etiqueta *hideNavBar*.

```

<Drawer key={Routing.drawer} hideNavBar contentComponent={DrawerContainer}>
  <Scene
    key={Routing.home}
    title={strings('title.home')}
    component={HomeContainer}
    hideNavBar
  />
</Drawer>
<Scene key={Routing.userRecipes} component={UserRecipesContainer} navigationBarStyle={styles.navBarColor} />
<Scene key={Routing.userFavourites} component={FavouritesContainer} navigationBarStyle={styles.navBarColor} />
<Scene key={Routing.editUserRecipes} component={EditRecipeContainer} navigationBarStyle={styles.navBarColor} />
<Scene key={Routing.createNewRecipe} component={CreateRecipeContainer} navigationBarStyle={styles.navBarColor} />
<Scene key={Routing.createNewRecipeStep} component={CreateStepContainer} navigationBarStyle={styles.navBarColor} />

```

Figura 48: Escenas de navegación en Router.js

8.2.4 Estilo de las Peticiones

La aplicación *frontend*, es decir, la interfaz de usuario requiere cierta lógica para poder establecer conexiones con el *backend* y la base de datos y así permitir al usuario el realizar llamadas a los datos o modificarlos. Para esto, se ha configurado un archivo en el que se han establecido diferentes métodos que serán los que realicen las llamadas o peticiones a la API. Este archivo llamado **ApiCalls.js** se encuentra en la rute `/src/services/api`. Gracias a esto, se podrá llamar desde cualquier lugar de esta aplicación a alguna de estos métodos en función de la acción de comunicación que se quiera realizar. Empezaremos por ver un ejemplo de las funciones, como puede ser el de crear una receta.

En la Figura 49 se presenta el método *newRecipe* que establece una llamada al *backend* y se le proporciona suficiente información para que realice la creación de una nueva receta. Lo primero que se ve es como la función se declara como asíncrona, es decir, ya que estos métodos dependen de una respuesta externa, al ser asíncronos son capaces de ejecutarse y que el resto de la aplicación espere mediante “promesas” a que reciba una respuesta. Se puede observar los 3 parámetros que se le pasa a la función. Estos son, en este caso, el identificador del usuario que está realizando la petición, el token del usuario que se envía por seguridad para traspasar el *firewall* del *backend*, y un objeto de valores que son los establecidos por el usuario que tendrá la receta.

Lo siguiente que se observa dentro del cuerpo de la función es como se ejecuta un método *fetch*, que es con el que se establecerá la llamada a la API. En primer lugar, se le proporciona la ruta, siendo la raíz de esta una variable global del entorno de la aplicación, para facilitar el cambio en caso de movimiento en la dirección del *backend*. Mas tarde, se configura el método HTTP a utilizar, en este caso POST. A continuación, viene la cabecera de la petición, en donde se especifica el tipo del contenido que acepta, en este caso Json. Además, aquí se encuentra también el token de seguridad del usuario, con etiqueta de *Authorization*. Ya por último en lo que concierne a la petición, viene el cuerpo de ésta. Aquí se codifica en Json un objeto Javascript con todos los valores pasados por el parámetro de *values*.

Por último, gracias a la promesa *.then*, se establece que, una vez completada y procesada la petición, el método retornará la respuesta en formato Json.

```

export const newRecipe = async (userId, userToken, values) => (
  fetch(`${process.env.API_KEY}/api/recipe/new`, {
    method: 'POST',
    headers: {
      'Accept': 'application/json',
      'Content-Type': 'application/json',
      'Authorization': `Bearer ${userToken}`,
    },
    body: JSON.stringify({
      userId,
      title: values.title,
      text: values.text,
      rating: 0,
      steps: values.steps,
      img: values.img,
      prepTime: values.prepTime,
      difficulty: values.difficulty,
      people: values.people,
      ingredients: values.ingredients,
    }),
  }).then((response) => response.json())
    .catch((error) => error) // to catch the errors if any;
);

```

Figura 49: Función de llamada a la API para la creación de una receta

8.3 Pruebas

A medida que se iban desarrollando los diferentes módulos tanto del *frontend* como del *backend* se fueron realizando pruebas funcionales y de usabilidad. Además, al llegar a las fases finales del desarrollo, se probó el funcionamiento global de toda la aplicación, tanto de forma casual como forzando situaciones adversas o inesperadas para ver como respondía la aplicación y lo que debía esperar el usuario. También se realizaron estas pruebas en diferentes dispositivos *hardware* de *Android*, además de en varios emuladores diferentes con variedad de resoluciones y distribución *Android*. Respecto al *backend*, gracias a la aplicación de *Postman* se pudieron realizar pruebas de llamadas con diferentes campos personalizados, tokens, etc. Gracias a estas pruebas, se fueron descubriendo fallos en ciertas funcionalidades o *bugs* inesperados dentro de la aplicación que podrían haber comprometido su funcionamiento, lo que ayudó a sacar una versión más limpia y segura.

Gracias a estas pruebas, se corroboró el correcto funcionamiento de:

- El usuario puede ver de manera correcta el listado y la información de recetas.
- La búsqueda se realiza de forma correcta.
- El usuario es capaz de crear sin ningún problema recetas nuevas para la aplicación.
- Los comentarios, añadir a favoritos y votación de recetas funcionan según lo esperado.
- El sistema elimina y edita las recetas con seguridad.
- Inicios de sesión y registros correctos.
- Visualización de las recetas propias del usuario y favoritos funciona correctamente.
- Buen flujo de navegación entre pantallas.
- Seguridad del sistema garantizada.

9. Conclusiones y trabajo futuro

9.1 Resultados y Conclusiones

Tras terminar con el desarrollo e implementación del proyecto, se ha acabado como resultado con la aplicación funcional ajustada en gran manera con la visión de la empresa descrita en las primeras fases de planteamiento y diseño de la idea.

Una vez terminado el proyecto en su totalidad, tanto lo que entra en el apartado del desarrollo de la aplicación como de la redacción de su informe, el autor es capaz de mirar atrás y reflexionar sobre cómo ha sido uno de los proyectos más extensos en los que ha trabajado, aportando horas de esfuerzo en diferentes ámbitos durante el periodo de un semestre académico. La experiencia que ha aportado este trabajo ha sido bastante extensa y enriquecedora, dando al autor una gran cantidad de experiencia en todos los ámbitos de desarrollo que caben en un proyecto de tal envergadura, desde las primeras fases de planificación, pasando por el diseño y desarrollo y el plasmar todo el conjunto en un informe detallado.

Desde el inicio se ha acogido con ilusión, gracias al gran apoyo de empresa y tutores por igual, que ofrecía una oportunidad de formación en tecnologías y procesos de desarrollo nunca vistos por el alumno, necesarios para el ámbito laboral en este sector tecnológico como es el desarrollo de tecnologías móviles.

Este proyecto también ha servido como reto personal y para ampliar aquellos campos de conocimiento en el que el alumno no había tratado con anterioridad en la universidad, y, en los casos en los que sí se parte con conocimientos previos, ha servido para ponerlos en práctica en un entorno de trabajo real. Por ejemplo, el desarrollo del *backend* del proyecto ha servido como experiencia en aprender como funciona la capa de conexión de aplicaciones y servicios en aplicaciones reales, cosa que no se trata en profundidad en asignaturas de desarrollo de aplicaciones web, y ayuda a aplicar el conocimiento de redes a las conexiones con peticiones y respuestas.

Tras un largo desarrollo, que ha presentado sus altos y sus bajos, el proyecto llega a su fin y ha dejado una huella en el autor que, a la larga, solo recordará las buenas experiencias que le ha aportado, y sabrá aplicar las enseñanzas en su día a día con soltura y confianza.

9.2 Dificultades encontradas

Durante el desarrollo del proyecto no se han encontrado por lo general muchas dificultades. Las principales que caben destacar son la falta de conocimiento del alumno de las tecnologías propuestas por la empresa. A pesar de que se contaba con experiencia en desarrollo web y tecnologías de desarrollo de aplicaciones móviles híbridas como *Angular*, el autor no presentaba trabajo previo en las herramientas de *React* o *react native*. Tampoco se había trabajado nunca con tecnologías de *backend*. A pesar de que se habían desarrollado previamente aplicaciones web con el *framework* de PHP *Laravel* en las prácticas de empresa cursadas por el alumno, no se tenía maestría en el manejo del *framework* requerido por la empresa aplicado a APIs. Todo esto, aunque ayuda al alumno a ganar experiencia en estas tecnologías, implica que la velocidad de desarrollo se ve algo comprometida ya que se debe ir aprendiendo a medida que se desarrollan funcionalidades, aunque se parte de la base de tecnologías conocidas parecidas.

Otra dificultad que destacar encontrada al inicio de la fase de desarrollo fue la configuración del *framework* para que se adaptase al proyecto base proporcionado por la empresa, ya que presentaba varios problemas de compatibilidad entre versiones de dependencias, y hubo que realizar un estudio de éstas para poder continuar con el desarrollo, lo que restó bastante tiempo de éste.

9.3 Mejoras Futuras

Como todo proyecto software, si quiere mantenerse a flote en el mercado de software requiere un flujo constante de nuevos diseños y mejoras que implementar. Además, a lo largo de la fase de desarrollo, algunas funcionalidades pensadas de la aplicación tuvieron que ser descartadas para versiones futuras y así poder adaptarse correctamente al marco de tiempo establecido. Algunas de estas funcionalidades son:

- Capacidad del usuario de subir videos o álbumes de imágenes en las recetas.
- Un sistema de categorías para el filtrado de las recetas.
- Sistema de verificación de cuenta por correo electrónico o teléfono móvil.
- Sistema geolocalización para establecer países de la receta y poder explorar recetas típicas del extranjero.
- Mejoras en la visualización e interacción del usuario con la aplicación.
- Sistema avanzado de búsqueda.
- Ranking de las recetas más votadas de los usuarios.

10. Manual de usuario

A continuación, se procederá a describir desde el punto de vista del usuario cual sería el comportamiento que debería seguir dentro de la aplicación.

Una vez el usuario se haya descargado la aplicación y la lance, lo primero que verá será la pantalla de carga con el *splashscreen*. Una vez la aplicación haya cargado se presentará la **pantalla principal** (Figura 50), un listado deslizable hacia abajo con las recetas creadas por los usuarios.



Figura 50: Pantalla principal

Desde esta pantalla, el usuario podrá realizar una búsqueda sencilla al presionar en el icono de la lupa y escribir una palabra clave, lo que cambiará el contenido del listado por las recetas que coincidan:



Figura 52: Búsqueda de recetas



Figura 51: Resultado de la búsqueda

Si el usuario encuentra en el listado principal o tras la búsqueda una receta que quiera ver con más detalle solo tiene que pulsar en ella y se abrirá una nueva ventana con la información detallada: foto de la receta, foto del usuario que lo ha creado, *rating* en formato 5 estrellas, tiempo de preparación, dificultad, descripción, autor, ingredientes y pasos de la preparación. Los pasos tienen a su vez una foto, el número de paso, nombre y descripción.

 Caldo de papas







 40 min

 Fácil

Pariatur eveniet voluptatum unde nisi. Ab id quis suscipit ab quae impedit qui. Error possimus quo voluptatem doloribus quas. Ipsum qui amet laborum esse quis commodi sed. Cumque id occaecati esse officia....

Autor:
Abuelita de prueba

Ingredientes

- 2 cucharadas de sal
- 5 papas

Preparación



1

Pelar las papas
Primero lavamos y pelamos las papas



2

Preparar el caldo
Mezclamos las papas y el resto de ingredientes en una olla con agua hirviendo

Figura 53: Pantalla de detalles de la receta

Además, al final de la receta se ve la sección de comentarios, compartir y puntuar, lo que abrirá un modal diferente para cada una de las opciones. Al no estar registrado, le pide al usuario iniciar sesión o registrarse para poder dejar un comentario:



Figura 54: Información final de la vista detallada de la receta



Figura 55: Modal de puntuación de receta



Figura 56: Modal de compartir por medios externos

Si el usuario no cuenta con una cuenta en la base de datos deberá registrarse. Para ello accede desde el menú desplegable izquierdo en la pantalla principal, donde se dirigiría al apartado de registrarse. Aquí aparece un formulario con los diferentes campos a rellenar: foto de perfil (opcional), nombre y apellidos, email, contraseña y confirmar contraseña. Además, el formulario cuenta con validaciones para cuando los campos no cumplen los requisitos, la contraseña confirmada está mal, o si el email ya se encuentra en uso. Este último va en función de la respuesta del *backend*.



Figura 58: Menú desplegable (Drawer)

A registration form titled "Registrarse" with a back arrow. The text "Bienvenido! Proporcionanos los siguientes datos para darte de alta" is displayed. There is a circular placeholder for a profile picture and a yellow button labeled "Añadir foto de Perfil". Below are four input fields: "Nombre y apellidos", "Email", "Contraseña", and "Repetir contraseña". At the bottom is a large yellow button labeled "Crear cuenta".

Figura 57: Formulario de registro

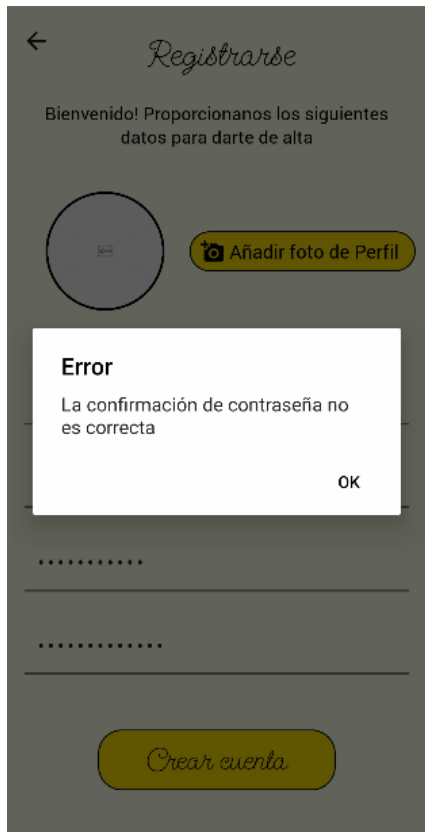


Figura 60: Ejemplo de fallo de la validación de registro



Figura 59: Ejemplo del formulario de registro completado

Si el usuario ya posee una cuenta creada con anterioridad, procede a iniciar sesión. Del mismo modo que el registro, desde el *drawer*, pulsará en iniciar sesión y será llevado al formulario de inicio de sesión. Aquí de la misma manera ponemos el correo y contraseña y nos valida en caso de que alguno este mal.



Figura 61: Inicio de sesión

Una vez dentro de su sesión, hay algunos cambios en lo que ya se ha mostrado. El primero es que en el *drawer* ahora se muestra la foto de perfil y el nombre, además de varias opciones. Estas son: “Mi Perfil” (Figura 63), donde se podrá ver y editar la foto y nombre, “Favoritos” (Figura 62), “Nueva Receta” (Figura 71), “Mis Recetas” (Figura 66) y salir de la sesión (Figura 65).

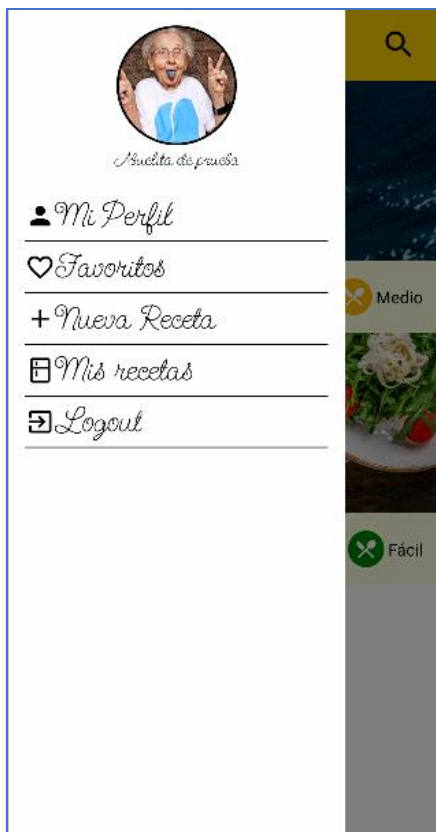


Figura 64: Drawer con sesión iniciada



Figura 63: Editar Perfil

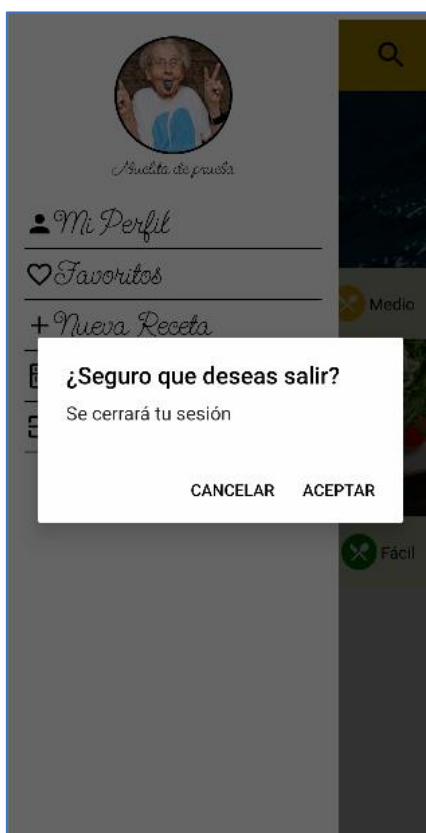


Figura 65: Cerrar sesión



Figura 62: Favoritos



Figura 66: Recetas del usuario

Al acceder a una receta con la sesión iniciada, se pueden ver también algunos cambios. Primero se muestra un corazón en la esquina superior derecha de la receta, donde al pulsar se añade a favoritos y el icono cambia. Además, en el final de la receta se puede añadir un comentario.

Comentarios
Abuelita de prueba
 Está muy rico :)

Escribe aquí para añadir un comentario

Añade un comentario

¡He dicho!

Figura 67: Dejar comentario

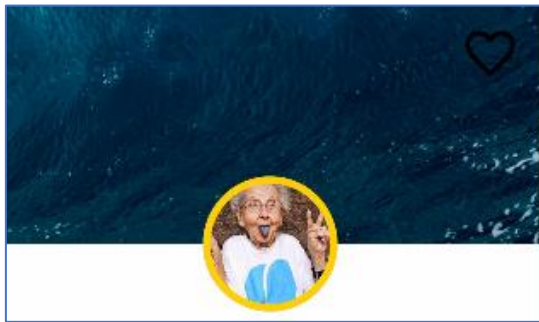


Figura 69: Corazón de favoritos

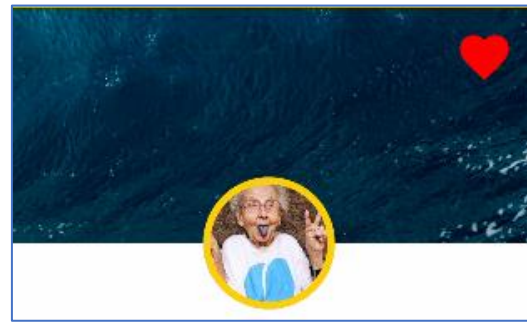


Figura 68: Receta añadida a favoritos

Ahora vamos veremos cómo crear las recetas. Desde el *drawer* se accede a "Nueva Receta" y se abrirá un formulario de 3 pantallas para crear la receta. Cada pantalla tiene sus formularios y validaciones correspondientes. La primera tiene información general, la segunda listado de ingredientes, que se pueden añadir y eliminar y número de personas. La tercera tiene el listado de los pasos. Los pasos a su vez tienen una pantalla propia donde se añade la imagen del paso, el título y la descripción. Esta lista es ordenable arrastrando los pasos, lo que actualiza el número. Además, se puede editar y eliminar cada entrada. Cosas como las imágenes son opcionales y se pone una por defecto.

Figura 71: Paso 1 creación de receta

Figura 70: Paso 2 creación de receta



Figura 73: Paso 3 creación de receta



Figura 72: Creación de paso nuevo

La edición de una receta hará uso de estas vistas, pero con los campos rellenos con la información perteneciente a dicha receta. Al acceder desde el *drawer* a "Mis Recetas", se ve una pantalla parecida a la normal pero que con las recetas del usuario que se encuentre de alta en ese momento, y en las tarjetas sencillas de información se observan dos opciones adicionales: editar y eliminar. Editar lleva a la misma pantalla de crear, solo que, con los datos de la receta en los campos correspondientes para ser editados, y eliminar salta con un mensaje de si está el usuario seguro de querer borrar la receta.



Figura 74: Vista de "Mis Recetas"



Figura 75: Formulario de edición de receta

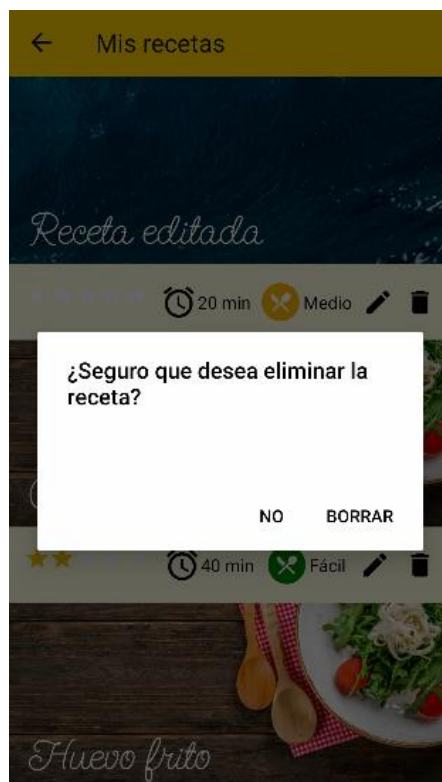


Figura 76: Mensaje de confirmación de borrado de receta

11. Referencias

[1] Growth from Knowledge - GfK. “Según GfK, sólo el 27% de los españoles siente pasión por la comida y por cocinar” [En línea]. 7 de mayo de 2015.

Disponible en:

<https://www.gfk.com/es/prensa/segun-gfk-solo-el-27-de-los-espanoles-siente-pasion-por-la-comida-y-por-cocinar>

[2] Martínez Dantí, Carla. “Realfooding: qué es y en qué consiste”, [En línea]. 18 febrero 2022.

Disponible en:

<https://www.mundodeportivo.com/uncomo/comida/articulo/realfooding-que-es-y-en-que-consiste-51931.html>

[3] Google Play Store. “Google Play Store”, [En línea].

Disponible en: <https://play.google.com/store/apps>

[4] Cookpad Inc. UK (2022) Cookpad (2.248.2.0) [Aplicación móvil]. Google Play.

<https://play.google.com/store/apps/details?id=com.mufumbo.android.recipe.search>

[5] MYREALFOOD APP, S.L. (2022) MyRealFood (4.6.30). [Aplicación móvil]. Google Play.

<https://play.google.com/store/apps/details?id=es.myrealfood.myrealfood>

[6] BuzzFeed (2022). Tasty (1.62.0), [Aplicación móvil]. Google Play.

<https://play.google.com/store/apps/details?id=com.mufumbo.android.recipe.search>

[7] Escuela de Ingeniería Informática (2022). “Objetivos y Competencias del GII”, [En línea].

Disponible en:

https://www2.ulpgc.es/archivos/plan_estudios/4008_40/ObjetivosyCompetenciasdelGII.pdf

[8] IONOS Digital Guide “El modelo en cascada: desarrollo secuencial de software” [En línea]. 11 de marzo de 2019.

Disponible en:

<https://www.ionos.es/digitalguide/paginas-web/desarrollo-web/el-modelo-en-cascada/>

[9] Terreros, Daniela. “Qué es Trello, para qué sirve y cómo funciona” [En línea]. 10 de Marzo de 2022.

Disponible en: <https://blog.hubspot.es/marketing/que-es-trello>

[10] NoCodeHackers. “Figma – Herramienta no-code de diseño” [En línea].

Disponible en: <https://www.nocodehackers.es/herramientas-no-code/figma>

[11] La Madriguera Bit. “¿Qué es Postman?” [En línea].

Disponible en: <https://lamadriguerabit.com/articulos/que-es-postman/>

[12] Microsoft (2022). “Microsoft Visual Studio (Versión 1.67)” [En línea].

Disponible en: <https://visualstudio.microsoft.com/es/>

[13] Google (2022). “*Android Studio (Versión Chipmunk)*” [En línea].
Disponible en: <https://developer.android.com/studio>

[14] GitHub Inc. (2022). “*GitHub*” [En línea].
Disponible en: <https://github.com/>

[15] EDUCBA. “*What is XAMPP?*” [En línea].
Disponible en: <https://www.educba.com/what-is-xampp/>

[16] García de Zúñiga, Fernán. “¿*Qué es phpMyAdmin y cómo usarlo?*” [En línea]. 25 de noviembre de 2021.
Disponible en: <https://www.arsys.es/blog/phpmyadmin>

[17] MariaDB. “*MariaDB*” [En línea].
Disponible en: <https://mariadb.org/>

[18] DB Designer. “*DB Designer*” [En línea].
Disponible en: <https://www.dbdesigner.net/>

[19] Microsoft. “*Office 365*” [En línea].
Disponible en: <https://www.office.com/>

[20] Brave Inc. “*Brave Browser*” [En línea].
Disponible en: <https://brave.com/es/>

[21] Git. “*Git*” [En línea].
Disponible en: <https://git-scm.com/>

[22] PHP. “*PHP*” [En línea].
Disponible en: <https://www.php.net/manual/es/intro-what-is.php>

[23] Mozilla Web Docs. “*Javascript*” [En línea].
Disponible en: <https://developer.mozilla.org/es/docs/Web/JavaScript>

[24] npm Inc. “*Node.JS?*” [En línea].
Disponible en: <https://nodejs.org/es/about/>

[25] npm Inc. “*What is npm?*” [En línea]. 26 de agosto de 2011
Disponible en: <https://nodejs.org/en/knowledge/getting-started/npm/what-is-npm/>

[26] Blanes, José Antonio. “¿*Qué es React Native?*” [En línea].
Disponible en: <https://www2.deloitte.com/es/es/pages/technology/articles/que-es-react-native.html>

[27] Statista. “*Cross-platform mobile frameworks used by software developers worldwide from 2019 to 2021*” [En línea]. 21 de febrero de 2022
Disponible en: <https://www.statista.com/statistics/869224/worldwide-software-developer-working-hours/>

- [28] Orange. “¿Qué son las licencias de ‘software’ y qué tipos hay?” [En línea]
Disponible en: <https://blog.orange.es/consejos-y-trucos/que-son-las-licencias-de-software-y-que-tipos-hay/>
- [29] Producingross. “Elegiendo una licencia” [En línea]
Disponible en: <https://producingoss.com/es/license-choosing.html>
- [30] Jefatura del Estado. BOE-A-2018-16673 [En línea] 6 de diciembre de 2018
Disponible en: <https://www.boe.es/buscar/pdf/2018/BOE-A-2018-16673-consolidado.pdf>
- [31] Lucidchart. “Qué es el lenguaje unificado de modelado (UML)” [En línea]
Disponible en: <https://www.lucidchart.com/pages/es/que-es-el-lenguaje-unificado-de-modelado-uml>
- [32] Aprendiendo a Programar. “Arquitectura cliente-servidor: definición, tipos, ejemplos, ventajas y desventajas” [En línea]. 22 de marzo de 2021
Disponible en: <https://aprendiendoaprogramar.es/blog/arquitectura-cliente-servidor-definicion-tipos-ejemplos-ventajas-y-desventajas/>
- [33] Lamentees Maravillosa. “La Ley de Hick y la toma de decisiones” [En línea].
Disponible en: <https://lamenteesmaravillosa.com/la-ley-de-hick-y-la-toma-de-decisiones/>
- [34] Klein, André. “Reglas de oro para diseñar interfaces” [En línea]. 29 de septiembre de 2016
Disponible en: <https://www.docpath.com/art-golden-rules-of-user-interface-design/?lang=es>
- [35] Symfony Docs. “Installing & Setting up the Symfony Framework” [En línea].
Disponible en: <https://symfony.com/doc/current/setup.html>
- [36] RedHat, Inc. “¿Qué es una API de REST?” [En línea]. 8 de mayo de 2020
Disponible en: <https://www.redhat.com/es/topics/api/what-is-a-rest-api>
- [37] Izertis. “Mejores prácticas para su API REST” [En línea]. 6 de octubre de 2017
Disponible en: <https://www.izertis.com/es/-/blog/mejores-practicas-para-su-api-rest>
- [38] Symfony Docs. “Databases and the Doctrine ORM” [En línea]
Disponible en: <https://symfony.com/doc/current/doctrine.html#:~:text=Symfony%20provides%20all%20the%20tools,also%20NoSQL%20databases%20like%20MongoDB.>
- [39] Lexik. “LexikJWTAuthenticationBundle (Versión 2.15.1)” [En línea].
Disponible en: <https://github.com/lexik/LexikJWTAuthenticationBundle/blob/2.x/Resources/doc/index.rst#getting-started>
- [40] FriendsOfSymfony. “FOSRestBundle (Versión 3.3.0)” [En línea].
Disponible en: <https://github.com/FriendsOfSymfony/FOSRestBundle>

- [41] Symfony Docs. “Forms” [En línea]
Disponible en: <https://symfony.com/doc/current/forms.html>
- [42] Priya Pedamkar. “Introduction to PHP Annotations” [En línea]
Disponible en: <https://www.educba.com/php-annotations/>
- [43] React Docs. “File Structure” [En línea]
Disponible en: <https://reactjs.org/docs/faq-structure.html>
- [44] aksonov. “react-native-router-flux (Versión 4.3.1)” [En línea].
Disponible en: <https://github.com/aksonov/react-native-router-flux>
- [45] React Redux. “React Redux” [En línea].
Disponible en: <https://react-redux.js.org/>
- [46] localz. “react-native-searchbar” [En línea].
Disponible en: <https://github.com/localz/react-native-searchbar/releases>
- [47] react-native-image-picker. “react-native-image-picker (Versión 4.8.4)” [En línea].
Disponible en: <https://github.com/react-native-image-picker/react-native-image-picker>
- [48] oblador. “react-native-vector-icons (Versión 9.1.0)” [En línea].
Disponible en: <https://github.com/oblador/react-native-vector-icons>
- [49] fnando. “i18n-js” [En línea].
Disponible en: <https://github.com/fnando/i18n-js>
- [50] djchie. “react-native-star-rating” [En línea].
Disponible en: <https://github.com/djchie/react-native-star-rating>
- [51] react-native-modal. “react-native-modal (Versión 13.0.1)” [En línea].
Disponible en: <https://github.com/react-native-modal/react-native-modal>
- [52] jeanregisser. “react-native-slider (Versión 0.11.0)” [En línea].
Disponible en: <https://github.com/jeanregisser/react-native-slider>
- [53] computerjazz. “react-native-draggable-flatlist (Versión 3.1.1)” [En línea].
Disponible en: <https://github.com/computerjazz/react-native-draggable-flatlist>
- [54] crazycodeboy. “react-native- splash-screen (Versión 3.3.0)” [En línea].
Disponible en: <https://github.com/crazycodeboy/react-native-splash-screen>
- [55] Growth from Knowledge - GfK. “Estudio global: Cocina: actitudes y tiempo que los consumidores emplean en ella” [En línea]. Abril de 2015.
Disponibles en:
https://cdn2.hubspot.net/hubfs/2405078/cms-pdfs/fileadmin/user_upload/dyna_content/es/reports/rep-2015-q2-informe_global_gfk_cocina.pdf
- [56] Symfony Docs - GfK. “Databases and Doctrine (“The Model”)” [En línea].

Disponible en: <https://symfony2-document.readthedocs.io/en/latest/book/doctrine.html>